Swarm and Evolutionary Computation 78 (2023) 101282

Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo

Function value ranking aware differential evolution for global numerical optimization

Dong Liu [a,b], Hao He [a,b], Qiang Yang [c,*], Yiqiao Wang [d], Sang-Woon Jeon [e], Jun Zhang [e]

[a] College of Computer and Information Engineering, Henan Normal University, Xinxiang, PR. China
[b] Key Laboratory of Artificial Intelligence and Personalized Learning in Education of Henan Province, Xinxiang, PR. China
[c] School of Artificial Intelligence, Nanjing University of Information Science and Technology, Nanjing, PR. China
[d] School of Management, Beijing University of Chinese Medicine, Beijing, PR. China
[e] Department of Electrical and Electronic Engineering, Hanyang University, Ansan, Korea

ARTICLE INFO

Keywords:
Differential evolution
Global numerical optimization
Function value ranking aware differential evolution
Mutation operation
Multimodal problems

ABSTRACT

Differential evolution (DE) has been experimentally demonstrated to be effective in solving optimization problems. However, the effectiveness of DE encounters rapid deterioration in the face of complicated optimization problems with many grotesque local basins, which are emerging increasingly frequently nowadays. To alleviate this predicament, this paper devises a simple yet effective mutation scheme named "DE/current-to-rwrand/1" to further promote the optimization ability of DE in solving complicated optimization problems. Specifically, this strategy assigns a non-linear selection probability for each individual, which is computed based on its function value ranking. As a result, all individuals can be potentially selected to direct the mutation of the population, but better individuals preserve exponentially larger selection probabilities. In this way, the resultant DE, which is named function value ranking aware differential evolution (FVRADE), is expectedly capable of balancing high diversity and fast convergence of the population well to find satisfactory solutions to optimization problems. In particular, two classical and popular mutation strategies, namely "DE/current-to-rand/1" and "DE/current-to-best/1", are two special cases of the proposed mutation strategy. Abundant experiments have been extensively executed on the well-known CEC'2017 and the latest CEC'2021 benchmark problem sets. Experimental results have verified that FVRADE performs highly competitively with or even significantly better than several representative and state-of-the-art peer methods. In particular, it is experimentally demonstrated that FVRADE is particularly good at solving complicated optimization problems, and preserves a good scalability to deal with optimization problems. Besides, experiments have also been conducted on the popularly adopted CEC'2011 real-world optimization problem suite and the experimental results have substantiated that FVRADE is very promising to solve real-world optimization problems.

1. Introduction

Differential evolution (DE) has witnessed rapid development and great success in solving optimization problems [1,2] since its first emergence [3]. In DE, a set of individuals, each of which represents a feasible solution to the optimization problem, are maintained to iteratively seek the global optimum [3]. During the search process, individuals are evolved by three main operations, namely the mutation operation, the crossover operation, and the selection operation [3–6]. Since individuals can search for optimal solutions in diversified directions, DE usually preserves strong global search ability [7–9]. By means of this unique advantage, DE has been widely employed to cope with a variety of optimization problems, for instance, multimodal problems [5,9,10], multi-objective problems [2,11], and expensive optimization problems [12,13]. Besides, DE has also been widely utilized to tackle a lot of real-world optimization problems, for example protein structure prediction [14] and 5 G communication [15].

However, with the advancement of Internet of Things (IoT), real-world optimization problems become more and more complex [16,17]. On the one hand, more and more variables are involved in optimization problems, and these variables are usually accompanied by intricate interactions [16,18]. As a consequence, the landscape of the

Abbreviations: DE, Differential Evolution; FVRADE, Function Value Ranking Aware Differential Evolution; GNO, Global Numerical Optimization.

* Corresponding author.
E-mail address: mmmyq@126.com (Q. Yang).

https://doi.org/10.1016/j.swevo.2023.101282
Received 12 June 2022; Received in revised form 28 January 2023; Accepted 14 February 2023
Available online 16 February 2023
2210-6502/© 2023 Elsevier B.V. All rights reserved.

optimization problem becomes grotesque, which brings considerable difficulty to problem optimization. On the other hand, the solution space may contain an increasingly large number of local basins and saddle areas [19–21]. Consequently, stagnation and premature convergence are generally encountered during problem optimization. These two challenges seriously deteriorate the optimization effectiveness and efficiency of DE.

To deal with the above challenges, researchers have been devoted to designing effective and efficient DE, and therefore numerous remarkable DE variants have emerged [5,8,22,23]. As far as we are concerned, existing researches on DE mainly proceed in two directions, namely designing novel mutation strategies to effectively evolve individuals [8,9,22,24,25], and devising adaptive parameter strategies to alleviate the sensitivity of DE to two key parameters [22,26,27].

In DE, the mutation operation exerts significant influence on assisting DE to obtain promising performance [24,25,28–30]. Therefore, numerous contributions have been made to devising effective mutation strategies to evolve the population. At first, by utilizing different population topology structures, various mutation mechanisms were designed to mutate individuals based on exchanged information [29–32]. Then, to fully utilize historically evolutionary information, various mutation schemes incorporating historical evolutionary information were devised to direct the mutation of individuals [28,30,33–35]. Furthermore, since different mutation strategies preserve different advantages and strengths in exploring and exploiting the solution space, some researchers even proposed to assemble different mutation operators to evolve the population [5,7–9]. As far as we know, there are three main ways of assembling different mutation methods, namely, 1) simultaneously utilizing multiple mutation strategies to generate multiple offspring and then selecting the best one as the final offspring [7,36–38], 2) simultaneously utilizing multiple mutation strategies to generate a weighted offspring [4,39,40], and 3) adaptively selecting one proper mutation strategy for each individual to generate its offspring [41–44].

Except for the mutation operator, two key parameters in DE, namely the scaling parameter denoted as F in the mutation operator and the probability parameter represented as CR in the crossover operator, also make significant influence on the optimization performance of DE [26,27,45–47]. In particular, the optimal settings of these two parameters are usually different for DE on different optimization problems [26,27]. This leads to that DE is very sensitive to these two parameters. To alleviate this issue, researchers have been devoted to devising adaptive parameter adjustment strategies for these two parameters, so that tedious and time-consuming efforts to fine-tune them on different problems can be saved. As a result, a large number of effective adaptive strategies have emerged [9,26,27,32,41]. In general, the core idea behind these adaptive strategies is to dynamically adjust the settings of these two parameters based on captured evolutionary information of the population [41,46]. In a broad sense, existing adaptive parameter strategies can be divided into two main types: 1) the population-level adaptive strategies [45,47,48], where all individuals share the same settings of these two parameters and such settings are dynamically regulated according to the evolutionary information of the population and the individuals, and 2) the individual-level adaptive strategies [9,32,46], where each individual has its own settings of the two parameters and such settings are adaptively regulated according to the evolutionary information of individuals.

Although the research on DE has been advanced a lot since its first advent, the optimization performance of DE still encounters a variety of challenges especially when dealing with complicated optimization problems, which increasingly emerge with the development of IoT [16]. In particular, premature convergence and stagnation often occur when individuals in DE search the complex solution space of optimization problems with many correlated variables and a lot of local basins and saddle regions [19,20].

To alleviate the above predicament, this paper proposes a simple yet effective mutation scheme, named "DE/current-to-rwrand/1", for DE to effectively tackle complicated optimization problems. Specifically, this strategy first computes selection probabilities of individuals in the current population based on a non-linear weight function related to their function value rankings after they are sorted from the best to the worst. In this manner, better individuals have exponentially larger selection probabilities, while worse individuals preserve exponentially lower chances to be chosen as the guiding exemplars to evolve the population. Subsequently, based on the computed selection probabilities, the proposed mutation strategy randomly selects a guiding exemplar from the current population to mutate each individual based on the roulette wheel section method. By equipping this mutation strategy with an existing adaptive parameter adjustment strategy for F and CR, a new DE variant, which is named function value ranking aware differential evolution (FVRADE), is developed.

To summarize, the novelty of this paper is listed below:

1) A simple yet effective mutation scheme, called "DE/current-to-rwrand/1", is devised. Different from most existing mutation strategies [22,29,31,32], which only adopt the absolutely best individuals as the guiding exemplars to update the population, the proposed mutation method employs an exponential weight function to calculate the selection probability of each individual based on its function value ranking after all individuals are sorted in the ascending order of their function values. With this weight function, better individuals preserve exponentially larger selection probabilities, while worse individuals own exponentially smaller selection probabilities. In this way, on the one hand, fast convergence could be expectedly guaranteed since the population is likely guided by better individuals; on the other hand, high population diversity could be hopefully maintained since each individual can be potentially employed to evolve the population. Therefore, the devised mutation strategy is expected to help DE make a good compromise between search diversity and convergence to mine the problem space properly.
2) In particular, it is found that two classical and popular mutation strategies, namely "DE/current-to-best/1" [32], and "DE/current--to-rand/1" [8] are two special cases of the devised "DE/current--to-rwrand/1". Specifically, when one parameter in the weight calculation of individuals is set as a very small value, "DE/current-to-rwrand/1" degenerates into "DE/current-to-best/1"; By contrast, when a very large value is assigned to the parameter, it degrades into "DE/current-to-rand/1".

To substantiate the effectiveness of FVRADE, experiments are extensively carried out on the commonly adopted CEC'2017 benchmark set [49] and the latest CEC'2021 benchmark suite [50] by comparing FVRADE with state-of-the-art and representative peer approaches. Besides, in-depth investigations on FVRADE are also extensively conducted on the CEC'2017 benchmark set to find what contributes to its good performance. In addition, the effectiveness of FVRADE in solving real-world optimization problems is also verified by conducting experiments on the CEC'2011 benchmark set [51] which contains various optimization problems in real-world applications.

The rest parts of this article are organized as follows. The basic DE and its latest and representative variants are reviewed in Section 2. Then, Section 3 elaborates FVRADE in detail, following which is the verification of its effectiveness by extensive experiments in Section 4. At last, conclusions and discussions on future work are offered in Section 5.

2. Related work

2.1. Basic differential evolution

In general, DE maintains a set of individuals to iteratively seek satisfactory solutions to optimization problems [1,22,23]. With NP

individuals maintained in the population, the overall procedure of the classical DE is shown in Algorithm 1.

In the beginning, DE first randomly initializes *NP* individuals according to the uniform distribution in the following way, which is widely used in the literature for evolutionary algorithms [52–56]:

$$x_i^d = LB^d + \mathrm{rand}(0,1) \times \left(UB^d - LB^d\right) \tag{1}$$

where $\boldsymbol{x}_i = [x_i^1, x_i^2, \ldots, x_i^d, \ldots, x_i^D]$ is the *i*th individual containing *D* variables and $i \in \{1, \ldots, NP\}$ is the index of an individual, $[LB^d, UB^d]$ is the search range of the *d*th variable, and rand (0,1) generates a real number randomly sampled within [0,1].

After the initialization of the population, the algorithm goes into the main loop of the evolution, where the following three evolution operators are involved:

1) **Mutation:** The mutation operator is employed for constructing a mutation vector $\boldsymbol{v}_{i,t}$ for each parent individual $\boldsymbol{x}_{i,t}$. In the literature [22,25], a lot of mutation operations exist. In this section, this paper first lists several classical mutation strategies below and then reviews the latest ones in the following:

"DE/rand/1" [9]

$$\boldsymbol{v}_{i,t} = \boldsymbol{x}_{rand0,t} + F \times \left(\boldsymbol{x}_{rand1,t} - \boldsymbol{x}_{rand2,t}\right) \tag{2}$$

"DE/best/1" [5]

$$\boldsymbol{v}_{i,t} = \boldsymbol{x}_{best,t} + F \times \left(\boldsymbol{x}_{rand1,t} - \boldsymbol{x}_{rand2,t}\right) \tag{3}$$

"DE/current-to-best/1" [27]

$$\boldsymbol{v}_{i,t} = \boldsymbol{x}_{i,t} + F \times \left(\boldsymbol{x}_{best,t} - \boldsymbol{x}_{i,t}\right) + F \times \left(\boldsymbol{x}_{rand1,t} - \boldsymbol{x}_{rand2,t}\right) \tag{4}$$

"DE/current-to-pbest/1" [32]

$$\boldsymbol{v}_{i,t} = \boldsymbol{x}_{i,t} + F \times \left(\boldsymbol{x}_{pbest,t} - \boldsymbol{x}_{i,t}\right) + F \times \left(\boldsymbol{x}_{rand1,t} - \boldsymbol{x}_{rand2,t}\right) \tag{5}$$

"DE/current-to-rand/1" [8]

$$\boldsymbol{v}_{i,t} = \boldsymbol{x}_{i,t} + F \times \left(\boldsymbol{x}_{rand1,t} - \boldsymbol{x}_{i,t}\right) + F \times \left(\boldsymbol{x}_{rand2,t} - \boldsymbol{x}_{rand3,t}\right) \tag{6}$$

In the above formula, *rand*0, *rand*1, *rand*2, and *rand*3 are mutually different individual indexes uniformly sampled from {1, …, *NP*}, and they also differ from *i*, namely $rand0 \neq rand1 \neq rand2 \neq rand3 \neq i$. $\boldsymbol{x}_{best,t}$ is the best one among all individuals in the current population. *F* is a scaling parameter controlling the step size of learning from the difference vectors. $\boldsymbol{x}_{pbest,t}$ is randomly selected from the top *p* best individuals in the population.

2) **Crossover**: The crossover operator is conducted to recombine the parent individual $\boldsymbol{x}_{i,t}$ and the associated mutation vector $\boldsymbol{v}_{i,t}$ to form the final offspring, which is also called the trial vector $\boldsymbol{u}_{i,t}$. In the literature, the most widely used crossover strategy is the binomial crossover by means of its easiness in implementation and high diversity [5,28,36,40,57]. Specifically, the binomial crossover recombines the parent individual $\boldsymbol{x}_{i,t}$ and the associated mutation vector $\boldsymbol{v}_{i,t}$ in the following manner to generate the final offspring:

$$u_{i,t}^d = \begin{cases} v_{i,t}^d, \text{if} d == d_{rand} \quad \text{or} \quad \mathrm{rand}(0,1) \le CR \\ x_{i,t}^d, \text{otherwise} \end{cases} \tag{7}$$

where $u_{i,t}^d$ is the *d*th variable of the trail vector $\boldsymbol{u}_{i,t}$ associated with the *i*th parent individual $\boldsymbol{x}_{i,t}$ in the *t*th iteration. d_{rand} is a dimension index randomly chosen from [1, *D*], which is used to ensure that at least one element of the generated offspring $\boldsymbol{u}_{i,t}$ is different from the associated parent individual $\boldsymbol{x}_{i,t}$; rand(0,1) is a real number generator, which randomly and uniformly samples a real number within [0,1]. *CR* is the crossover probability, which takes charge of the difference between the generated offspring and the associated parent. Therefore, *CR* significantly affects the optimization performance of DE.

3) **Selection:** After the generation of the offspring, the selection operator is conducted to compare the generated offspring $\boldsymbol{u}_{i,t}$ and the associated parent $\boldsymbol{x}_{i,t}$ [58]. Then, the better one survives for the next evolution as follows:

$$\boldsymbol{x}_{i,t+1} = \begin{cases} \boldsymbol{u}_{i,t}, \text{if} \quad f\left(\boldsymbol{u}_{i,t}\right) < f\left(\boldsymbol{x}_{i,t}\right) \\ \boldsymbol{x}_{i,t}, \text{otherwise} \end{cases} \tag{8}$$

With the iteration of the above three operators, individuals in DE are continuously updated to traverse the problem space until the termination condition is met. Finally, the best solution found by the population is taken as the output.

2.2. Advancement of DE

To elevate the optimization ability of DE in solving optimization problems, as far as we know, researchers mainly concentrate their attention on devising novel techniques for DE from the following two perspectives.

2.2.1. Researches on mutation strategies

With the advancement of DE, it has been widely recognized that it is the mutation operation that exerts significant impact on the optimization performance of DE [25,44,59,60]. As a result, in recent years, researchers have poured extensive attention into designing effective and efficient mutation strategies for DE to achieve satisfactory optimization performance [9,31,43,55,57].

In the beginning, by organizing individuals in the population with different topology structures, researchers have proposed many different individual selection methods to select parent individuals participating in the mutation operation, leading to different mutation strategies for DE [22,29–32]. For instance, different from "DE/best/1" where each individual only interacts with the best one in the population, Sarker et al. [29] proposed to let individuals interact with the top best $0.1 \times NP$ to $0.5 \times NP$ ones in the population. Specifically, they randomly selected an individual from these best individuals as the base vector to direct the mutation of each individual. In this way, individuals in the population could be mutated in more diversified directions, and thus high diversity could be maintained. Likewise, Sun et al. [31] also improved "DE/best/1" by first randomly selecting five different individuals from the top $0.2 \times NP$ best ones and then utilizing the best one among the five individuals as the base vector to direct the mutation of each parent. In addition, instead of only using the best one among all individuals in the population to direct the mutation of others in "DE/current-to-best/1", "DE/current-to-pbest/1" [32] was proposed by utilizing a random one of the top ranked $\lfloor p \times NP \rfloor$ $(0<p<1)$ individuals in the current population to mutate each individual. In [61], the authors first proposed a mutation method to enhance the exploration ability of individuals, which is called "DE/current-to-ord_best/1". Specifically, for each target individual to be mutated, this mutation strategy randomly selects three different individuals from the current population and then sorts them from the best to the worst to direct the mutation of the target individual. Then, they further designed another mutation scheme to promote the exploitation ability of individuals, which is named "DE/current-to-ord_pbest/1" with two individuals randomly selected from the current population, while the other chosen from the top $\lfloor p \times NP \rfloor$ $(0<p<1)$ best ones. In [53], Gao et al. proposed a chaotic local search method and then let it cooperate

with "DE/current-to-pbest/1" to tackle optimization problems. With this strategy, the resultant DE could well balance exploration and exploitation during the optimization.

In order to further enhance the diversity of the population, some researchers have proposed to select parent individuals in the mutation operation based on neighborhood information [40,59,62]. For instance, in [40], a neighborhood and direction information based DE (NDi-DE) was devised by designing a neighbor based selection scheme to choose parent individuals used in the mutation operator and a directional mutation mechanism to mutate individuals. With these mechanisms, the neighborhood and direction information can be fully employed to guide individuals to more promising areas. In [62], Cai et al. proposed a learning-enhanced DE (LeDE) by first partitioning the population into several clusters and then devising a clustering learning strategy to update individuals in the clusters. For individuals in the same cluster, an intra-cluster learning strategy was designed for exchanging information in the same cluster. For individuals in different clusters, an inter-cluster learning strategy was designed to exchange information between different clusters. By this means, information in the same cluster and between different clusters can be fully used to direct the mutation of individuals. To effectively solve multimodal problems, a neighborhood mutation strategy [59] was proposed by only allowing individuals to interact with their own Euclidean neighborhoods to mutate. Likewise, in [60], a proximity based mutation scheme was proposed by first maintaining an affinity matrix based on Euclidean distance between individuals and then selecting parent individuals involved in the mutation operator according to the matrix to direct the mutation of individuals. By virtue of this mechanism, the neighborhood information can be incorporated to direct the evolution of the population and thus high diversity could be maintained during the evolution without seriously sacrificing the convergence. Analogously, Cai et al. [63] proposed a self-organizing neighborhood based mutation strategy by utilizing the self-organizing map with the cosine similarity to determine neighbors of individuals. Besides, they also designed an adaptive strategy to dynamically adjust the neighbor sizes of different individuals based on their evolutionary states. With this strategy, the neighborhood information among individuals can be fully employed to evolve the population.

Except for the function value based individual selection strategies, in recent years, some researchers have proposed many other measures to cooperate with the function value to design novel individual selection methods for DE. For instance, Xia et al. [55] proposed a novelty-hybrid-fitness driving DE (NFDDE) by selecting parent individuals involved in the mutation operation based on the function values of individuals and a newly defined "novelty" measure. Based on the function value, it randomly selects an exemplar from the top best p elites in the population to construct a difference vector. Next, on the basis of the novelty measure, which is defined as the average distance between one individual and its k nearest neighbors, it randomly selects an exemplar from the first highest p individuals in the current population to generate another difference vector. With these two difference vectors, the generated mutation vectors are expected to scatter in diverse areas. In this way, high diversity and fast convergence are expected to be better balanced during the evolution. In [64], Cheng et al. proposed a function value and diversity ranking-based mutation operator for DE. In brief, they first designed a diversity estimation strategy based on the function values of individuals to estimate the diversity contribution of each individual. Then, they sorted all individuals by considering both the objective ranking and the diversity contribution of each individual. Based on the new rankings of individuals, parent individuals involved in the mutation operator are randomly selected. In [65], Zhao et al. proposed an archive based mutation strategy based on the "top-bottom" mechanism. With this new mutation scheme, historical heuristic information gained from the successful and failed individuals can be fully utilized to guide individuals toward promising regions. Gong et al. [66] proposed an objective value ranking-based mutation strategy for DE. In this mutation scheme, individuals are sorted from the best to the worst and then the selection probability of each individual is computed based on its ranking. Based on the calculated selection probabilities, individuals involved in the mutation operation are randomly selected based on the roulette wheel selection method. Zou et al. [67] focused on utilizing the information of unsuccessful individuals to direct the selection of parent individuals in the mutation operation. To be specific, based on the number of the consecutive iterations where each individual encounters unsuccessful updates, the selection probabilities of all individuals are computed. Then, based on such probabilities, the base individual and the terminal individual in the mutation operation are randomly selected based on the roulette wheel selection strategy. In [68], a triangular mutation operator was devised by first randomly selecting three individuals from the population and then combining the three difference vectors between any two of the three selected individuals to mutate each individual. In this way, it is expected that a better balance between global exploration and local exploitation can be achieved during the optimization process.

Subsequently, to fully utilize the potentially valuable information embedded in obsolete individuals, some researchers have even proposed various individual selection schemes by incorporating historically evolutionary information [28,32,34]. To name a few representatives, Zhang *et al.* [32] maintained an archive to keep the obsolete parent individuals and subsequently combined them with the current population together to randomly choose individuals to construct the random difference vector for each individual in the mutation operator. In this way, more diversified random directions could be afforded to mutate individuals. Meng *et al.* [69] maintained a big archive to store the historical population in the past several generations and then employed these historical individuals along with the current population to generate a random difference vector for each individual. In this way, historical information in the continuously past generations is fully utilized to evolve the population. Likewise, in [33], Meng *et al.* kept two kinds of archives to store historical information. The first archive is the one in [32] to store the obsolete parent individuals, while the second is the one in [28] used to store the historical population in the previous generations. Then, they utilized the first archive along with the current population to generate a random difference vector and used the second archive along with the current population to generate another random difference vector for each individual. In [35], an archive was maintained to store the best individuals found during the evolution and then cooperated with DE to deal with dynamic optimization problems. Different from the above archive mechanisms which store historical individuals, Ghosh et al. [34] maintained an archive to store the random difference vectors associated with the successful offspring which are better than their parents. Then, they reused these difference vectors adaptively to generate offspring during the evolution. In this way, promising directions in past generations can be used to generate offspring.

Empirically, different mutation strategies own different advantages and strengths in exploring and exploiting the solution space. Therefore, some researchers have attempted to assemble multiple mutation strategies to evolve the population [4,5,9,36,42,43]. To the best of our knowledge, there are three main ways of assembling multiple mutation methods, which are reviewed in the following.

1) Simultaneously utilizing multiple mutation strategies to generate multiple offspring and then selecting the best one as the final offspring [7,37,38]. For instance, a composite DE (CoDE) [36] was proposed by simultaneously using three mutation schemes to generate three mutation vectors for each individual. Therefore, this DE variant generates three offspring for each individual in each generation. In the selection process, these three offspring are compared with their parent, and the best one survives. In [1], Yi et al. adopted two mutation strategies to generate two offspring for each individual. Then, these two offspring are compared with the parent

individual, and the best one enters the next generation. In [37], a cumulative population distribution information based DE (CPI-DE) was devised by maintaining two different coordinate systems. Specifically, this algorithm first constructs an Eigen coordinate with the covariance matrix adaptation according to the cumulative information of the population. Then, for each target individual, one offspring is constructed in the original coordinate and another offspring is built in the Eigen coordinate. Then, in the selection operation, these two offspring are compared with their parent, and the best one enters the next generation. In [38], Gong *et al.* first adopted multiple mutation strategies to generate a group of offspring for each individual. Then, they designed a cheap surrogate model to select the best one from the offspring group as the final offspring of the associated parent individual. Similarly, in [7], Zhou *et al.* also utilized multiple mutation strategies to generate a set of candidate offspring simultaneously. Then, they devised a cheap abstract convex underestimation model to select the most suitable candidate as the final offspring of each parent individual. In this way, all mutation operators participate in the offspring generation and a lot of function evaluations are saved to evolve the population.

2) Simultaneously utilizing multiple mutation strategies to build a weighted mutation vector and then using this mutation vector to generate one offspring [4,39,40]. For example, Das et al. [4] proposed to hybridize two different mutation strategies, namely a local mutation model and a global mutation model, to generate one offspring for each individual. Specifically, they used a weight factor to linearly combine the two mutation vectors generated by the two mutation operators and then adopted the weighted mutation vector to crossover with the associated parent to get one offspring. In [40], Cai et al. proposed an information fusion strategy to combine three types of difference vectors in different directions. Specifically, the first direction difference vector is the directional attraction, which is the difference between the individual and its best near-neighbor individual; the second is the directional repulsion, which is the difference between the individual and its worst near-neighbor individual; the third is the directional convergence, which is the combination of the former two difference vectors. These three directional vectors are combined by a newly designed direction-inducing scheme to generate an offspring for each individual. In [39], a two-stage mutation strategy was devised by first generating a mutation vector for each individual based on a neighborhood-based mutation scheme, and then, further mutating the generated mutation vector by linearly combining it with the local best individual via a weight factor.

3) Adaptively selecting proper mutation strategies to generate offspring [9,41,42]. For instance, in [5], five different mutation strategies are maintained as a candidate mutation pool. Then, these five mutation strategies are all utilized to generate offspring in the first generation. Hereafter, the best one among the five operators is selected based on the optimization results in the first generation and then is used to evolve the population until the iteration terminates. However, the selection of the mutation strategy is not suitable because it is only based on the optimization results in one generation. In [44], four different mutation strategies are adopted to evolve the population. Specifically, during the evolution, each mutation method is used to mutate a group of individuals in the population, and the size of the group mutated by each mutation strategy is adaptively adjusted based on the quality of the generated offspring. Likewise, Zhan et al. [43] also used different mutation strategies to evolve different subgroups of individuals. Specifically, during the evolution, individuals in the population are first divided into superior ones, normal ones and inferior ones according to their objective values. Subsequently, "DE/current-to-pbest/1" is employed to mutate superior individuals to accelerate convergence, while "DE/current-to-rand/1" is adopted to mutate inferior individuals to maintain high diversity. For normal individuals, a random one is selected from the above two strategies according to the estimation of the evolutionary state of the population. In [41], an individual-dependent mutation strategy was devised to mutate individuals. Specifically, individuals in the current population are first partitioned into superior ones and inferior ones at different stages of the evolution process. Then, four different mutation methods with different advantages are employed to mutate the two kinds of individuals, respectively. Different from the above methods, which adaptively select the appropriate mutation strategy at the group level, some researchers have designed adaptive selection methods to select the proper mutation strategy at the individual level. For instance, in [42], a mutation strategy pool containing several different mutation strategies is first maintained. Then, during the evolution, each individual randomly selects a mutation scheme from the pool based on the selection probabilities of these mutation strategies, which are computed based on the quality of the corresponding generated offspring. To further improve the selection quality of the mutation strategies, Liu et al. [9] first maintained three different mutation strategies and then they combined the historical experience of the population and the heuristic information of each individual to choose an appropriate mutation strategy for each individual to conduct the mutation. Besides, in [70], the authors first decomposed all variables into several groups and then used two different DE variants to cooperatively optimize the decomposed variable groups. In [23], Xia *et al.* proposed a fitness-based adaptive DE (FADE), which first divides the population into multiple small subgroups and then assigns different mutation strategies to evolve different subgroups of individuals according to their own characteristics. Similarly, in [71], the population is first divided into multiple small groups, and then each individual within each group randomly selects a proper mutation strategy and a suitable control parameter from the maintained mutation pool and the maintained control parameter setting pool. In [72], a DE variant with dual mutation strategies collaboration (DMCDE) was proposed. To be specific, a variant of "DE/rand/2" and a variant of "DE/best/2" were developed to evolve individuals adaptively, so as to obtain a good trade-off between global exploration and local exploitation.

Although the above mentioned mutation strategies have assisted DE to achieve promising performance in coping with various optimization problems, their effectiveness encounters great challenges in tackling complicated optimization problems. To alleviate this issue, this paper designs a simple yet effective mutation strategy, called "DE/current-to-rwrand/1", for DE by assigning a probability to each individual based on its objective value ranking.

2.2.2. Researches on adaptive parameter adjustment

In DE, the scaling parameter denoted by F in the mutation operator and the probability parameter denoted by CR in the crossover operator also play key roles in helping DE obtain good performance [26,27,45–47]. Specifically, it is experimentally found that on the one hand, the most suitable settings of these two parameters for DE with the same mutation and crossover operations are usually different on different optimization problems [26,27]; on the other hand, on the same optimization problem, the settings of the two parameters in different DEs with different mutation strategies are also usually different [26,27]. Since finding the optimal settings of these two parameters for DE is very tedious and time-consuming, researchers have been devoted to designing adaptive parameter adjustment strategies to alleviate the sensitivity of DE to these two parameters [9,26,27,32,41]. As far as we know, existing adaptive parameter control approaches could be classified into two main types, which are described below:

1) Population-level adaptive methods [26]. In this category of adaptive methods, all individuals share the same settings of the two parameters and then during the evolution, the settings of these two parameters are adaptively regulated according to the evolutionary

information of the population [48]. Along this line, the most typical method is the linear adaptive scheme designed by Das *et al.* [45], where the scaling factor F gradually decreases linearly from a maximum value to a minimum value as the evolution goes on. In [47], a fuzzy adaptive DE was devised by utilizing fuzzy logic controllers to adaptively regulate F and CR.

2) Individual-level adaptive methods [26]. In this category of adaptive methods, every individual has its own settings of the two parameters, and the settings of different individuals are usually different. Along this line, the most representative method is the one proposed in JADE [32]. Specifically, it first generates the settings of F and CR according to two probability distribution models in the following way:

$$\begin{aligned} CR_i &= randn(\mu_{CR}, 0.1) \\ F_i &= randc(\mu_F, 0.1) \end{aligned} \tag{9}$$

where CR_i and F_i are the crossover probability and the scaling factor of the ith individual, respectively. $\text{randn}(\mu_{CR}, 0.1)$ is the Gaussian distribution by setting the mean with μ_{CR} and the standard variance with 0.1. $\text{randc}(\mu_F, 0.1)$ is the Cauchy distribution by setting the position parameter as μ_F and the scaling parameter as 0.1. In the initialization stage, μ_{CR} and μ_F are both set as 0.5. Then, they are adaptively updated every generation based on the recorded successful CR and F values associated with the successful offspring which are better than their parents and replace them to enter the next generation. In particular, μ_{CR} and μ_F are updated in the following way:

$$\begin{aligned} \mu_{CR} &= (1-\beta) \times \mu_{CR} + \beta \times \text{mean}_A(\boldsymbol{S}_{CR}) \\ \mu_F &= (1-\beta) \times \mu_F + \beta \times \text{mean}_L(\boldsymbol{S}_F) \\ \text{mean}_L(\boldsymbol{S}_F) &= \frac{\sum_{F \in \boldsymbol{S}_F} F^2}{\sum_{F \in \boldsymbol{S}_F} F} \end{aligned} \tag{10}$$

where β is a constant and is usually set as 0.1 [32]. $\boldsymbol{S}_{CR}$ and $\boldsymbol{S}_F$ are the set of all successful CR_i and F_i, respectively. mean_A denotes the arithmetic mean, while mean_L represents the Lehmer mean. In the above manner, the settings of F and CR are adaptively adjusted based on the evolutionary information of successful offspring [32].

Taking inspiration from JADE, many researchers have developed other adaptive parameter strategies for DE. For instance, in [30], Zhou et al. developed an improved version of JADE, named JADE_Sort, by sorting the generated CR values from the smallest to the largest, and then allocating smaller CR values to better individuals. In this way, the offspring of better individuals preserve smaller differences from their parents and thus, better individuals focus on exploiting the located optimal areas, while worse individuals concentrate on exploring the solution space. In [43], instead of equally averaging the successful CR and F values in JADE, Zhan et al. adopted the objective improvement between each successful offspring and its parent to calculate a weight and then utilized the weights to compute a weighted μ_{CR} and μ_F. In this way, the generated CR and F values bias to the ones which make greater contributions to the objective improvement of individuals. In [36], a two-level adaptive mechanism was proposed. In the upper level, F and CR for the whole population are adaptively regulated according to the evolution state of the population, which is estimated according to the population distribution. Subsequently, in the lower level, F and CR for each individual are further adaptively regulated according to its objective value and its distance to the global best solution. By this means, the evolutionary information of the population and the individuals are both taken into consideration to adaptively adjust the two key parameters. In [73], Wang et al. utilized two kinds of Cauchy distribution models, namely randc(0.65, 0.1) and randc(1, 0.1), to generate the setting of F, and another two kinds of Cauchy distribution models, namely randc(0.1, 0.1) and randc(0.95, 0.1), to generate the setting of CR for each individual. Specifically, in the initialization stage, a Cauchy distribution is randomly chosen to construct a value of F and another Cauchy distribution is also randomly chosen to build a value of CR for each individual. Then, during the evolution, if the constructed offspring outperforms its parent, the generated F and CR values are retained for the next iteration; otherwise, the settings of F and CR are regenerated by randomly selecting a Cauchy distribution from the associated candidates, respectively. In [41], an individual-dependent parameter strategy was devised to adaptively adjust the settings of F and CR accompanied with every individual. Particularly, two kinds of individual-dependent parameter control methods were designed. The first is the ranking-based scheme, which sets F and CR for every individual according to its objective value ranking after all individuals are sorted in the ascending order of their objective values. Another is the objective value-based scheme, which sets F and CR for each individual based on the difference in objective values. In [9], the historical experience information of successful offspring and the heuristic information of each individual with respect to its objective value ranking are both taken into consideration to adaptively adjust the settings of F and CR. In [74], Ghosh et al. put forward a nearest spatial neighborhood based adaptive parameter method for F and CR. To be specific, for each individual, the method first finds its several nearest neighbors in the population according to the Manhattan distance. Then, it employs the recorded successful F and CR associated with the neighbors of each individual to get the settings of the two parameters for the individual for the next iteration.

The above mentioned adaptive parameter strategies have been demonstrated to effectively aid DE to achieve very promising performance and thus in the literature [22,43], they have been frequently employed to cooperate with novel mutation strategies to help DE obtain good performance by researchers. In this paper, we also directly utilize the adaptive parameter control method in JADE_Sort [30] to cooperate with the proposed "DE/current-to-rwrand/1" to help DE solve optimization problems.

3. Proposed FVRADE

With the development of IoT, complicated optimization problems, such as those with many interacting variables and the ones with many local basins or saddle areas [75,76], have been becoming more and more common in real-world engineering [14–16]. To solve these complicated optimization problems, high diversity maintenance is the key for DE to fully search the solution space, so that optimal areas can be fast located. To this end, this paper proposes a simple yet effective mutation scheme, called "DE/current-to-rwrand/1", to improve the mutation diversity of individuals and accordingly a new DE variant, which is named function value ranking aware DE (FVRADE), is developed.

3.1. "DE/current-to-rwrand/1"

In the literature [1,28,72], it is empirically found that employing elites to guide the evolution of individuals is very promising for DE to obtain good performance. For instance, "DE/current-to-best/1" [27] and "DE/current-to-pbest/1" [32] have been widely employed to cooperate with other techniques to help DE effectively solve various optimization problems [12]. However, most of the existing elite guided mutation strategies [29,32,46] only adopt a few absolutely best ones among all individuals to evolve the population. On the one hand, once these few best individuals fall into local basins, the guided population is also at a great risk of falling into local regions, and thus misses finding the global optimum. This phenomenon likely occurs especially when DE is utilized

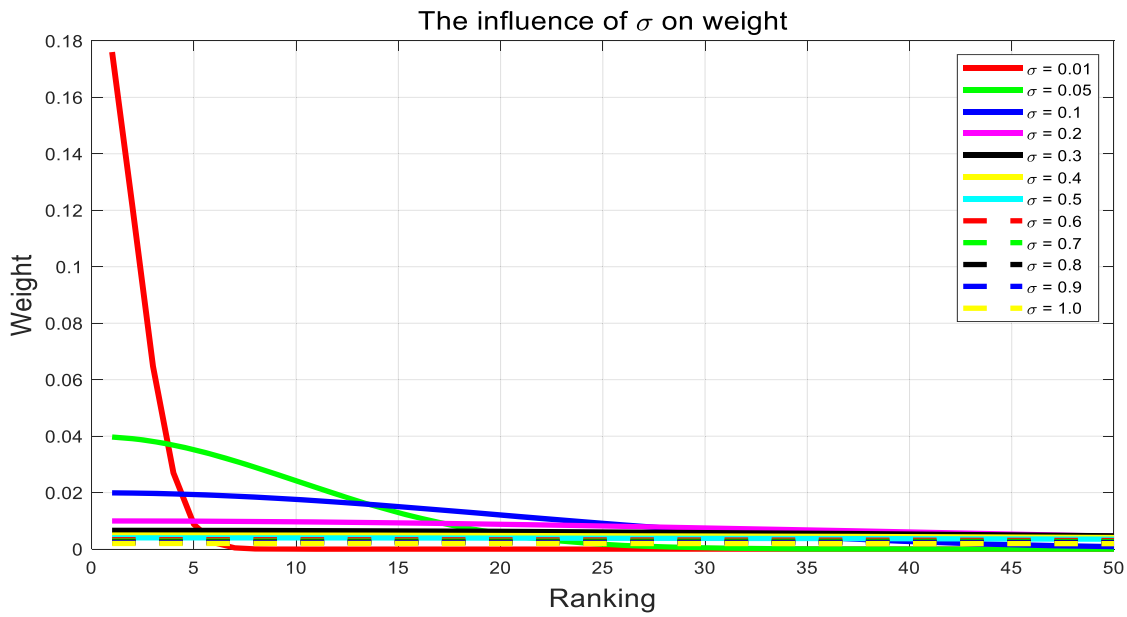

Fig. 1. Change of the weights of individuals with different settings of σ.

to tackle complex optimization problems with a lot of flat local or saddle areas; on the other hand, during the evolution, individuals in the population are mainly guided to search the solution space in very limited directions because they are directed by a very small number of elites. With these two potential limitations, DEs with existing elite guided mutation strategies [30,32,46] encounter optimization performance deterioration in front of increasingly complicated optimization problems nowadays.

To alleviate the above issue, a new mutation operation called "DE/current-to-rwrand/1" is designed in this paper. Specifically, instead of only using several top ranked individuals with respect to the objective values to direct the update of the population, "DE/current-to-rwrand/1" first allocates each individual a selection probability according to its objective value ranking, and thus each individual has a potential chance to be selected to direct the mutation of others. In this manner, individuals in the population can be mutated to search the solution space in various directions, and thus high diversity could be maintained during the evolution.

Specifically, after individuals in the current population are sorted in the ascending order of their objective values, the selection probability of each individual is calculated as follows:

$$p_i = \frac{w_i}{\sum_{j=1}^{NP} w_j} \tag{11}$$

where p_i denotes the selection probability associated with the ith individual, NP is the size of the population, and w_i is the selection weight of the ith individual, which is computed as follows:

$$w_i = \frac{1}{\sigma NP\sqrt{2\pi}} e^{-\frac{(rank(i)-1)^2}{2\sigma^2 NP^2}} \tag{12}$$

where $rank\ (i)$ indicates the objective value ranking of the ith individual, and σ is a parameter controlling the decreasing speed of the weight as the ranking increases.

Subsequently, based on the above computed selection probabilities, a guiding exemplar is randomly picked out from all individuals in the population based on the roulette wheel selection method to guide the mutation of each individual with the devised "DE/current-to-rwrand/1" in the following way:

$$\boldsymbol{v}_{i,t} = \boldsymbol{x}_{i,t} + F_i \times \left(\boldsymbol{x}_{i,rwrand,t} - \boldsymbol{x}_{i,t}\right) + F_i \times \left(\boldsymbol{x}_{rand1,t} - \boldsymbol{x}_{rand2,t}\right) \tag{13}$$

where $\boldsymbol{x}_{i,t}$ is the ith individual in the tth generation, $\boldsymbol{x}_{i,rwrand,t}$ is the guiding exemplar for the ith individual, which is randomly chosen from the population based on the above calculated selection probabilities. F_i is the scaling factor for the ith individual. $\boldsymbol{x}_{rand1,t}$ and $\boldsymbol{x}_{rand2,t}$ are two mutually different individuals with one randomly chosen from the current population $\boldsymbol{P}$, while another randomly chosen from $\boldsymbol{P} \cup \boldsymbol{A}$, where $\boldsymbol{A}$ is an additional archive of size NP, and is used to store obsolete parent individuals. It should be mentioned that to improve the search effectiveness, we let the better one between the two randomly selected individuals as $\boldsymbol{x}_{rand1,t}$, while the worse one as $\boldsymbol{x}_{rand2,t}$. That is, $\boldsymbol{x}_{rand1,t}$ is better than $\boldsymbol{x}_{rand2,t}$. In this way, the mutation vector is expectedly generated towards promising directions. The effectiveness of this directional difference is verified by the experiments conducted in Section 4.4.

In Eq. (13), the guiding exemplar $\boldsymbol{x}_{i,rwrand,t}$ for each individual is randomly chosen from all individuals in the current population based on the computed selection probabilities. Therefore, Eq. (11) and Eq. (12) have significant effect on the optimization performance of "DE/current-to-rwrand/1". Eq. (12) shows that the weight of each individual is exponentially related to its objective value ranking. Specifically, the weights of individuals exponentially decrease as the associated rankings increase. This indicates that better individuals have exponentially larger selection probabilities, while worse individuals preserve exponentially smaller selection probabilities.

In Eq. (12), the parameter σ significantly affects the selection probability of each individual. Hence, to better observe its influence, this paper plots the weight changes of individuals with different settings of σ as the rankings of individuals increase. Fig. 1 shows the weight changes of individuals with 12 different settings of σ along with NP set as 200. From this figure, the following observations can be obtained:

1) The smaller the setting of σ is, the faster the weight decreases as the ranking increases. In this situation, fewer and fewer better individuals preserve more and more weights, leading to that the selection of the guiding exemplar $\boldsymbol{x}_{i,rwrand,t}$ mainly concentrates on the top few best individuals. Therefore, individuals are guided to search the solution space in a few directions. This is beneficial for the population to exploit the promising regions and therefore fast

convergence is expectedly obtained. Particularly, if σ is set to an infinitesimal value, "DE/current-to-rwrand/1" degenerates to "DE/current-to-best/1".

2) The larger the value of σ is, the milder the weight of the individual declines as its objective value ranking grows. In this case, more and more top best individuals participate in the selection of the guiding exemplar $\boldsymbol{x}_{i,rwrand,t}$. As a result, individuals could be guided to search the solution space in more diversified directions. This is beneficial for the population to explore the solution space and thus high diversity could be guaranteed. Particularly, as the value of σ increases, the weights of individuals tend to be more and more uniform, leading to that the roulette wheel selection gradually becomes the uniform selection. In particular, if σ is set to an infinite value, "DE/current-to-rwrand/1" degenerates to "DE/current-to-rand/1".
3) As a whole, all individuals have chances to be selected as the guiding exemplars to evolve the population though the selection probabilities of worse individuals are very small. With this mechanism, "DE/current-to-rwrand/1" is expected to help DE achieve a good compromise between fast convergence and high diversity to find satisfactory solutions to optimization problems.

Based on the above analysis, to make a compromise between search intensification and diversification, this paper sets σ as 0.1, so that the resultant DE could explore the solution space and exploit the found promising regions appropriately.

At last, it deserves notice that the calculation of the selection probabilities is only related to the rankings of individuals, which range from 1 to *NP*, and thus the selection probabilities can be pre-computed before the iteration of the evolution. Therefore, "DE/current-to-rwrand/1" is very efficient.

3.2. Difference between "DE/current-to-rwrand/1" and existing similar mutation strategies

As far as we know, the most similar mutation strategies with "DE/current-to-rwrand/1" are "DE/current-to-best/1" [27], "DE/current--to-pbest/1" [32], and "DE/current-to-rand/1" [8]. Compared with these popular mutation schemes, "DE/current-to-rwrand/1" mainly differs from them in the following aspects:

1) **"DE/current-to-rwrand/1" assigns a selection probability to each individual, and thus all individuals can be potentially selected to direct the mutation of the population.** However, in "DE/current-to-best/1" [27], only the global best individual is adopted as the guiding exemplar, and thus all individuals share the same guiding exemplar. Therefore, in comparison to "DE/current--to-best/1", "DE/current-to-rwrand/1" hopefully preserves higher mutation diversity. Likewise, in "DE/current-to-pbest/1" [32], only a few top best individuals are used to direct the mutation of the population. As a consequence, individuals are guided to search the solution space in very limited directions. Hence, compared with "DE/current-to-pbest/1", "DE/current-to-rwrand/1" also expectedly preserves higher mutation diversity. As a whole, it is expected that "DE/current-to-rwrand/1" could help DE achieve more promising performance than the two compared mutation strategies. Experiments conducted in Section 4.4 verify the superiority of "DE/current-to-rwrand/1" to "DE/current-to-best/1" and "DE/current-to-pbest/1".
2) **In "DE/current-to-rwrand/1", though all individuals can be potentially selected as the guiding exemplars to mutate the population, they are treated differently.** Specifically, better individuals have exponentially higher chances, while worse individuals preserve exponentially smaller selection probabilities to be chosen as the guiding exemplars. However, in "DE/current-to-rand/1" [8], all individuals are treated in the same way, and they have the same probabilities to be selected as the guiding exemplars. Therefore, in comparison with "DE/current-to-rand/1", "DE/current-to-rwrand/1" is expected to converge faster to promising areas, and thus could help DE achieve much more promising performance. Experiments conducted in Section 4.4 verify the great dominance of "DE/current-to-rwrand/1" to "DE/current-to-rand/1".
3) In particular, "DE/current-to-rwrand/1" can be seen as a general mutation framework. With different settings of σ, it preserves different strengths in exploring and exploiting the solution space. Particularly, it is found that "DE/current-to-rand/1" and "DE/current-to-best/1" are two special cases of "DE/current-to-rwrand/1" with σ set to an infinite value and an infinitesimal value, respectively.

3. Complete FVRADE

In addition to "DE/current-to-rwrand/1", the resultant FVRADE utilizes the binomial crossover operator to recombine the dimensions of the target individual and the generated mutation vector to build the associated offspring. Besides, to alleviate the sensitivity to the two key parameters (namely *F* and *CR*) in the mutation operation and the crossover operation, FVRADE directly utilizes the adaptive parameter control method in JADE_Sort [30] to adaptively adjust the values of *F* and *CR* for every individual. Incorporating these techniques into the DE framework, the complete FVRADE is shown in Algorithm 2.

To be specific, in the beginning, as shown in Line 1, *NP* individuals are randomly created and their objective values are evaluated. After that, the selection probability of each individual is pre-calculated as shown in Line 3 since such calculation is only related to the rankings but not specific individuals. Subsequently, FVRADE enters the main iteration of evolution as shown in Lines 4 ~ 37. In the main loop, as shown in Line 6, a set of *CR* values and a set of *F* values, both of which are of size *NP*, are first generated based on the associated Gaussian distribution and the related Cauchy distribution, respectively. Then, the set of *CR* values are sorted from the smallest to the largest (Line 7). After that, individuals in the current population are sorted in the ascending order of their objective values (Line 8). After these preparations, individuals in the population are updated as shown in Lines 9 ~ 25. During the update of each individual, the associated *CR* value is first obtained based on its objective value ranking as shown in Line 10. Then, a guiding exemplar is randomly chosen from all individuals in the current population based on the selection probabilities (Line 11), and two other individuals are randomly chosen from the population and the combination of the current population and the archive (Line 12). Subsequently, the two random individuals are compared (Lines 13 ~ 15). Next, each individual is mutated based on "DE/current-to-rwrand/1" (Line 16) and the binomial crossover is executed to get the final offspring (Lines 18 ~ 24). After that, the selection operator is performed along with the update of the archive and the successful *CR* and *F* values (Line 26 ~ Line 35). The above operators are iteratively executed until the maximum function evaluations are exhausted. Finally, before the algorithm terminates, the global best position along with its objective value is taken as the output.

From Algorithm 2, with respect to the time complexity, without considering the time of function evaluations, in each generation, $O(NP)$ is needed to generate the set of *CR* values and another $O(NP)$ is needed to generate the set of *F* values. Then, $O(NP\times\log NP)$ is needed to sort the set of *CR* values and another $O(NP\times\log NP)$ is needed to sort all individuals. Subsequently, $O(NP\times D)$ is required for the mutation operation, and another $O(NP\times D)$ is required for the crossover operator. In the selection process, quadruple $O(NP)$ is needed to update the population, the archive, and the set of successful *CR* and *F* values. After the selection, $O(NP)$ is required to adaptively update the parameters of the two distribution models. On the whole, the overall time complexity of FVRADE is $O(NP\times D+NP\times\log NP)$, which indicates that there is no serious additional complexity burden as compared with the classical DE.

4. Experiments

This section mainly carries out experiments to comprehensively verify the effectiveness of FVRADE in solving optimization problems. To be specific, in Section 4.1, the experimental setup is first introduced, including the adopted benchmark problem sets and the selected compared algorithms. Then, Section 4.2 presents extensive comparisons between FVRADE and several state-of-the-art and representative DE variants including four latest LSHADE variants on the CEC'2017 benchmark set with four different settings of the dimensionality. Subsequently, in Section 4.3, experiments are conducted to compare FVRADE with the compared DE variants on the CEC'2021 benchmark set with two different settings of the dimensionality. Next, Section 4.4 takes deep investigations on FVRADE to investigate the usefulness of the devised mutation scheme and the used additional archive. At last, in Section 4.5, we further conduct experiments on real-world optimization problems selected from the CEC'2011 benchmark set to compare FVRADE with the compared DE methods, so as to verify the effectiveness of FVRADE in solving real-world optimization problems.

4.1. Experimental setup

To comprehensively demonstrate the effectiveness and efficiency of FVRADE, this paper adopts three latest and widely used benchmark problem sets to conduct experiments. The first one is the CEC'2017 benchmark problem set [49], which contains totally 29 problems with four categories, namely 2 unimodal optimization problems (F_1, F_3), 7 multimodal optimization problems (F_4-F_{10}), 10 hybrid optimization problems (F_{11}-F_{20}) and 10 composition optimization problems (F_{21}-F_{30}). In particular, the latter two kinds of optimization problems are far harder to solve than the former two kinds of problems. Such a benchmark problem set has been widely used in the literature to evaluate the optimization performance of evolutionary algorithms [77–80]. For further concrete information, please refer to [49]. To comprehensively assess the optimization performance of FVRADE, this paper adopts four different dimension sizes, namely 10-*D*, 30-*D*, 50-*D*, and 100-*D,* to configure the CEC'2017 problems. The second one is the CEC'2021 benchmark problem set [50]. It contains 10 basic problems, but it has 5 types of transformations, namely basic transformation (000), shift transformation (010), shift rot transformation (011), bias shift transformation (110) and bias shift rot transformation (111), to transform the 10 basic problems to optimization problems with different complexity. Therefore, each type contains 10 problems, and totally 50 problems exist in the CEC'2021 set. For detailed information of these optimization problems, please refer to [50]. Likewise, to comprehensively demonstrate the optimization performance of FVRADE, this paper adopts two different settings of the dimensionality, namely 10-*D* and 20-*D*, to configure the CEC'2021 benchmark problems. The last one is the CEC'2011 benchmark problem set [51]. This benchmark set contains many real-world optimization problems with different dimension sizes and properties. Such a benchmark set is utilized to verify the effectiveness of FVRADE in solving real-world optimization problems. However, since this paper focuses on solving bound-constrained optimization problems, we only select 6 real-world optimization problems with bound constraints (F_1, F_2, F_4, F_7, F_{12}, and F_{13}) from the CEC'2011 set to conduct experiments.

To make comparisons with FVRADE, this paper selects 9 state-of-the-art and representative DE methods as the compared peer methods. Specifically, they are JADE [32], CJADE [53], CUSDE [67], DiDE [69], AGDE [81], FADE [23], TPDE [57], MRDE [71], and DMCDE [72]. Except for JADE, the other 8 compared DE methods are selected because on the one hand they are the latest DE variants developed in recent three years; on the other hand, they all concentrate on developing novel mutation strategies to improve the optimization performance of DE, which is also the focus of this paper. In particular, CJADE [53], CUSDE [67], DiDE [69], and AGDE [81] developed only one mutation strategy to evolve the population, while FADE [23], TPDE [57], MRDE [71], and DMCDE [72] assembled multiple mutation mechanisms to evolve the population. These methods just cover the two main directions of the research on mutation as elucidated in Section 2.2.1. In addition, JADE [32] is selected because for one thing, the devised mutation strategy "DE/current-to-rwrand/1" in FVRADE shares some similarity with the mutation strategy "DE/current-to-pbest/1" in JADE; for another, FVRADE directly adopts the parameter adaption strategy for *F* and *CR* in JADE to solve optimization problems.

To further demonstrate the effectiveness of FVRADE, we also compare FVRADE with the latest LSHADE variants, which has won the competition on real-parameter single objective optimization held in the IEEE Congress on Evolutionary Computation (CEC). Specifically, jSO [79], LSHADE-cnEpSin [82] and LSHADE-SPACMA [77] are selected to compare with FVRADE, because they won the competition on real-parameter single objective optimization in IEEE CEC 2017. Besides, we also employ EBLSHADE [61], which is a latest variant of LSHADE and has been demonstrated to be more effective than the above three variants, to compare with FVRADE. Since these methods are the winners on the CEC'2017 benchmark problem set, we only compare FVRADE with them on this benchmark set.

To make fair comparisons, all algorithms are executed independently 51 times. Besides, this paper adopts the median, the mean, and the standard deviation (std) values over the 51 runs to assess the optimization performance of all algorithms. Furthermore, the Wilcoxon rank sum test at the significance level of α=0.05 is conducted to tell whether there is significant difference between FVRADE and each compared method on each optimization problem. At the same time, the Friedman test at the significance level of α=0.05 is also performed to tell the significance with respect to the difference between FVRADE and the compared methods in terms of the overall optimization performance on one whole benchmark problem set [83]. On the basis of the Friedman test results, we further conduct the Nemenyi post-hoc test to observe the significance degree with respect to the performance difference between FVRADE and each compared algorithm.

Based on the Wilcoxon rank sum test results, the mark "+", "=" and "-" at the top right corner of each p-value in the related tables indicate that the optimization performance of FVRADE is significantly superior to, equivalent to, and significantly inferior to the associated compared method on the corresponding problem, respectively. The rows denoted as "*w/t/l*" in the tables summarize the numbers of "+", "=" and "-", respectively. Besides, the rows denoted as "*Rank*" in the tables show the average ranks of all algorithms achieved by the Friedman test. The rows denoted as "*CD*" show the difference between the average rank of FVRADE and that of each compared method. The value after "*CD*" in the first column is the critical domain value of the Nemenyi post-hoc test at the significance level of α=0.05. If the difference value is larger than this critical value, it means that there is a significant difference between FVRADE and the associated compared method with respect to the overall optimization performance. Otherwise, there is no significant difference between FVRADE and the associated compared method.

At last, it is worth mentioning that FVRADE is implemented by MATLAB, and all experiments are executed on the same PC with 4 Intel (R) Core(TM) i5–3470 3.20 GHz CPUs, 4Gb memory and the Ubuntu 12.04 LTS 64-bit system.

4.2. Comparisons on the CEC'2017 benchmark problem set

4.2.1. Comparisons between FVRADE and state-of-the-art DE methods

This section mainly makes comparisons between FVRADE and the 9 compared DE variants to verify its effectiveness by carrying out experiments on the CEC'2017 benchmark set with the four dimension sizes. In the experiments, for fairness, the total number of function evaluations is set as 10,000×*D* with *D* denoting the dimension size for all algorithms. Besides, for fair comparisons, we first conduct experiments on the CEC'2017 benchmark set to fine-tune the settings of the population size

Table 1
Optimal settings of the population sizes (*NP*) for all optimization algorithms on the CEC'2017 benchmark set with the four dimension sizes.

D	Parameter	FVRADE	CUSDE (2022)	TPDE (2021)	CJADE (2021)	FADE (2020)	DMCDE (2020)	DiDE (2020)	MRDE (2019)	AGDE (2019)	JADE (2009)
10	*NP*	90	100	50	70	Initialized	240	100	Initialized as 3 × 25	30	100
30		180	100	100	90	as 3 × 25	330	300	Adaptively	60	120
50		250	100	100	100	Adaptively	300	500	Adjusted	100	150
100		350	100	100	150	Adjusted	300	1000		50	200

Table 2
Summarized comparison results between FVRADE and the 9 state-of-the-art compared algorithms with respect to "*w/t/l*" derived from the Wilcoxon rank sum test and the average rank obtained from the Friedman test on the CEC'2017 suite with the four dimension sizes.

Problem Set	Problem Property	Index	FVRADE	CUSDE	TPDE	CJADE	FADE	DMCDE	DiDE	MRDE	AGDE	JADE
CEC'2017–10*D*	Unimodal Problems	*w/t/l*	–	0/2/0	0/2/0	0/2/0	1/1/0	0/2/0	0/2/0	1/1/0	0/2/0	0/2/0
	Simple Multimodal Problems		–	4/3/0	4/3/0	0/7/0	5/2/0	4/3/0	4/3/0	6/1/0	4/3/0	2/5/0
	Hybrid Problems		–	6/2/2	4/5/1	6/2/2	10/0/0	4/5/1	2/6/2	4/2/4	5/2/3	8/0/2
	Composition Problems		–	6/3/1	9/1/0	2/6/2	2/4/4	5/2/3	3/6/1	2/3/5	3/2/5	3/4/3
	Overall		–	16/10/3	17/11/1	8/17/4	18/7/4	13/12/4	9/17/3	13/7/9	12/9/8	13/11/5
	Overall	*Rank*	4.36	6.88	6.91	4.88	6.03	6.22	4.98	**3.93**	4.95	5.84
	Overall	*CD*=0.87	–	**2.52**	**2.56**	0.52	**1.67**	**1.86**	0.62	0.43	0.59	**1.48**
CEC'2017–30*D*	Unimodal Problems	*w/t/l*	–	1/1/0	1/1/0	2/0/0	2/0/0	2/0/0	0/0/2	2/0/0	0/0/2	2/0/0
	Simple Multimodal Problems		–	4/1/2	3/2/2	3/2/2	4/2/1	5/1/1	4/1/2	4/1/2	4/1/2	4/1/2
	Hybrid Problems		–	5/5/0	7/2/1	9/1/0	6/3/1	5/4/1	3/5/2	5/3/2	9/1/0	10/0/0
	Composition Problems		–	7/2/1	6/3/1	9/1/0	6/2/2	7/2/1	7/2/1	5/2/3	7/2/1	8/2/0
	Overall		–	17/9/3	17/8/4	23/4/2	18/7/4	19/7/3	14/8/7	16/6/7	20/4/5	24/3/2
	Overall	*Rank*	**3.24**	5.50	6.09	6.24	6.38	6.34	3.28	5.05	6.50	6.38
	Overall	*CD*=0.87	–	**2.26**	**2.84**	**3.00**	**3.14**	**3.10**	0.03	**1.81**	**3.26**	**3.14**
CEC'2017–50*D*	Unimodal Problems	*w/t/l*	–	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0	0/0/2	2/0/0	2/0/0	2/0/0
	Simple Multimodal Problems		–	4/1/2	3/1/3	4/1/2	6/1/0	4/1/2	3/1/3	4/1/2	4/1/2	3/3/1
	Hybrid Problems		–	6/0/4	8/1/1	9/1/0	5/1/4	7/1/2	4/4/2	7/0/3	7/0/3	9/1/0
	Composition Problems		–	4/3/3	6/1/3	7/2/1	6/2/2	7/2/1	6/2/2	4/2/4	7/0/3	7/3/0
	Overall		–	16/4/9	19/3/7	22/4/3	19/4/6	20/4/5	13/7/9	17/3/9	20/1/8	21/7/1
	Overall	*Rank*	**3.52**	4.45	6.72	5.83	5.62	6.48	4.31	5.17	6.79	6.10
	Overall	*CD*=0.87	–	**0.93**	**3.21**	**2.31**	**2.10**	**2.97**	0.79	**1.66**	**3.28**	**2.59**
CEC'2017–100*D*	Unimodal Problems	*w/t/l*	–	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0	2/0/0
	Simple Multimodal Problems		–	5/0/2	4/0/3	4/1/2	7/0/0	5/0/2	5/0/2	5/0/2	5/0/2	5/0/2
	Hybrid Problems		–	9/0/1	7/1/2	8/2/0	6/1/3	8/1/1	5/1/4	7/1/2	9/0/1	8/2/0
	Composition Problems		–	5/2/3	6/3/1	7/2/1	7/2/1	8/2/0	8/1/1	7/2/1	8/1/1	8/2/0
	Overall		–	21/2/6	19/4/6	21/5/3	22/3/4	23/3/3	20/2/7	21/3/5	24/1/4	23/4/2
	Overall	*Rank*	**2.66**	7.10	6.31	4.38	5.31	6.03	5.10	5.03	7.03	6.03
	Overall	*CD*=0.87	-	**4.45**	**3.66**	**1.72**	**2.66**	**3.38**	**2.45**	**2.38**	**4.38**	**3.38**

for all algorithms, since it is well recognized that the setting of the population size is usually problem-dependent and makes great influence on the optimization performance of DE [3].

For CUSDE [67], TPDE [57], and DiDE [69], they were also tested on the CEC'2017 benchmark set with the four imension sizes. Therefore, for fairness, we directly utilize the settings of the population size in the associated papers. For FADE [23] and MRDE [71], their population sizes are adaptively adjusted during the evolution. Therefore, the settings of their population sizes do not need fine-tuning. For the other 4 compared algorithms (namely CJADE [53], DMCDE [72], AGDE [81] and JADE [32]) and FVRADE, we first roughly tune the population size with a large step size and then further fine-tune the population size with a small step size. Tables S1 ∼ S5 in the supplementary material present the fine-tuning results of the five algorithms on the CEC'2017 set with the four dimension sizes, respectively. Based on the fine-tuning results, the optimal settings of the population size on the 10-*D*, 30-*D*, 50-*D* and 100-*D* CEC'2017 benchmark sets for all algorithms are summarized in Table 1.

Based on the above fine-tuned population sizes of all methods, we conduct comparative experiments to compare FVRADE with the 9 state-of-the-art and representative DE variants on the CEC'2017 set with the four settings of the dimensionality. Tables S6 ∼ S9 in the supplementary material display the detailed optimization performance of FVRADE and the 9 compared DE methods and their comparison results on the 10-*D*, 30-*D*, 50-*D*, and 100-*D* CEC'2017 benchmark sets, respectively. To make a summary, Table 2 exhibits the summarized statistical comparison results between FVRADE and the 9 compared methods on the CEC'2017 suite with the four dimension sizes regarding "*w/t/l*" derived from the Wilcoxon rank sum test results, the average ranks "*Rank*" obtained from the Friedman test and the rank difference "*CD*" between FVRADE and each compared method for the Nemenyi post-hoc test. Besides, Figs. S1 ∼ S4 in the supplementary material exhibit the convergence behaviors of FVRADE and the 9 compared DE algorithms on the 10-*D*, 30-*D*, 50-*D* and 100-*D* CEC'2017 benchmark sets, respectively.

From Table 2, Tables S6 ∼ S9, and Fig. S1 ∼ Fig. S4 in the supplementary material, the following conclusions are attained from the comparison results between FVRADE and the 9 compared methods:

1) On the 10-*D* CEC'2017 benchmark suite, a) From the perspective of the Friedman test results, the average rank value of FVRADE is only slightly worse than MRDE, but is better than those of the other 8 compared methods. This indicates that FVRADE achieves the second best overall performance on the entire 10-*D* CEC'2017 benchmark set. b) According to the critical value of the Nemenyi post-hoc test, FVRADE shows significantly better overall performance than 5 compared methods (CUSDE, TPDE, FADE, DMCDE, and JADE) and achieves equivalent overall performance with the other 4 compared methods (CJADE, DiDE, MRDE, and AGDE) on the 10-*D* CEC'2017 benchmark suite. c) Regarding the Wilcoxon rank sum test results, FVRADE shows significantly better performance than the 9 compared DE methods on at least 8 problems, while the number of problems where FVRADE displays significantly better performance than them is far more than the number of problems where FVRADE displays significant inferiority to them. In particular, FVRADE is better than CUSDE, TPDE, FADE, DMCDE, MRDE, and JADE on more than 13 problems. d) In view of the optimization performance on different categories of optimization problems, on the two unimodal problems (F_1 and F_3), FVRADE achieves equivalent performance with the 9 compared DE variants. On the 7 simple multimodal problems (F_4-F_{10}), FVRADE is significantly superior to 7 compared methods on more than 4 problems, while there are no problems where FVRADE is inferior to the 9 compared methods. On the 10 hybrid problems (F_{11}-F_{20}), FVRADE shows no worse performance than the 9 compared methods on at least 6 problems. On the 10 composition problems (F_{21}-F_{30}), FVRADE shows significant superiority to CUSDE, TPDE and DMCDE on 6, 9 and 5 problems, respectively, and only presents significant inferiority to them on at most 3 problems, while it achieves highly competitive performance with the other 6 compared DE methods. e) In terms of the convergence comparisons shown in Fig. S1, at the first glimpse, FVRADE performs the best among all optimizers concerning the solution accuracy and the convergence speed on 5 problems (F_1, F_3, F_4, F_{11}, and F_{18}). On the rest benchmark problems, FVRADE also achieves much faster convergence along with higher solution quality than most of the 9 compared DE variants. f) As a whole, compared with the 9 compared DE variants, FVRADE shows highly competitive or even significantly better performance on the 10-*D* CEC'2017 benchmark suite.
2) On the 30-*D* CEC'2017 benchmark suite, a) With respect to the average rank, FVRADE ranks the first place among all methods. This signifies that FVRADE achieves the best overall optimization performance among all methods on the 30-*D* CEC'2017 suite. b) From the critical value of the Nemenyi post-hoc test, except for DiDE, FVRADE presents significantly better overall performance than the other 8 compared methods on the entire 30-*D* CEC'2017 suite. c) In view of "*w/t/l*", FVRADE performs significantly better than the 9 compared DE methods on at least 14 problems, while it only displays significantly worse performance than them on at most 7 problems. In particular, FVRADE is better than CJADE, AGDE and JADE on at least 20 problems. d) As for the optimization performance on different types of optimization problems, on the two unimodal problems (F_1 and F_3), FVRADE achieves significantly better performance than CJADE, FADE, DMCDE, MRDE and JADE, but obtains significantly worse performance than DiDE and AGDE. On the 7 simple multimodal problems (F_4-F_{10}), FVRADE shows no worse performance than the 9 compared methods on more than 5 problems, and only displays significantly worse performance than them on at most 2 problems. On the 10 hybrid problems (F_{11}-F_{20}), except for DiDE, FVRADE is much better than the other 8 compared methods on at least 5 problems and is much worse than them on no more than 2 problems. On the 10 composition problems (F_{21}-F_{30}), FVRADE shows significant superiority to the 9 compared methods on at least 5 problems and only presents significantly worse performance than them on at most 3 problems. e) Concerning the convergence comparisons shown in Fig. S2, at the first look, FVRADE performs the best among all optimizers concerning the convergence speed and the solution accuracy on 8 problems (F_5, F_7, F_8, F_{16}, F_{21}, F_{23}, F_{24}, and F_{26}). On the rest optimization problems, FVRADE also attains much better solutions along with faster convergence than most of the compared 9 methods. f) On the whole, on the 30-*D* CEC'2017 benchmark suite, FVRADE shows much better performance than the 9 compared DE variants. In particular, we find that FVRADE is much more effective than most of the 9 compared DE methods on the hybrid problems and the composition problems. This reveals that FVRADE is particularly effective in solving complicated optimization problems.
3) On the 50-*D* CEC'2017 benchmark suite, a) The Friedman test results show that FVRADE achieves the lowest rank among all optimization methods. Besides, its rank value is much smaller than those of the 9 compared methods. This reveals that FVRADE achieves the best overall performance on the whole 50-*D* CEC'2017 benchmark set. b) With respect to the critical value of the Nemenyi post hoc test, the overall optimization performance of FVRADE is significantly better than that of 8 compared DE variants. c) The Wilcoxon rank sum test results signify that FVRADE significantly outperforms the 9 compared DE methods on at least 13 problems, while it is significantly inferior to them on at most 9 problems. In particular, FVRADE is better than CJADE, DMCDE, AGDE, and JADE on more than 20 problems. d) In view of the optimization performance on different kinds of optimization problems, on the two unimodal problems (F_1 and F_3), except for DiDE, FVRADE performs significantly better than the other 8 compared DE variants. On the 7 simple multimodal problems (F_4-F_{10}), FVRADE achieves highly competitive performance with TPDE and DiDE, but shows significant superiority to the other 7 compared methods on more than 3 problems and presents significantly worse performance than them on at most 2 problems. On the 10 hybrid problems (F_{11}-F_{20}), FVRADE shows significantly better performance than CUSDE, TPDE, CJADE, DMCDE, MRDE, AGDE, and JADE on at least 6 problems and only displays worse performance than them on no more than 4 problems. Compared with the other 2 DE variants, FVRADE shows slightly better performance. On the 10 composition problems (F_{21}-F_{30}), FVRADE shows significant superiority to TPDE, CJADE, FADE, DMCDE, DiDE, AGDE, and JADE on at least 6 problems, and only presents significant inferiority to them on at most 3 problems, while it achieves highly competitive performance with the other 2 DE methods. e) Regarding the convergence comparisons shown in Fig. S3, FVRADE obtains significantly better solutions along with faster convergence than the 9 compared methods on 11 problems (F_5, F_7, F_8, F_{13}, F_{16}, F_{17}, F_{20}, F_{21}, F_{23}, F_{24} and F_{26}). On F_1, F_3, and F_{12}, FVRADE achieves very similar performance with one compared method, but shows much better performance in both the solution quality and the convergence speed than the other 8 compared methods. f) Comprehensively, on the 50-*D* CEC'2017 benchmark suite, FVRADE still shows significantly better performance than most of the 9 compared methods. In particular, FVRADE displays significant superiority to most compared methods on complicated problems, like the hybrid problems and the composition problems.
4) On the 100-*D* CEC'2017 benchmark suite, a) From the perspective of the average rank, FVRADE still ranks the first place among all optimization methods, and its rank value is much smaller than those of the 9 compared methods. This demonstrates that FVRADE achieves the best overall performance on the entire 100-*D* CEC'2017 benchmark set. b) According to the critical value of the Nemenyi post hoc test, the overall optimization performance of FVRADE is significantly superior to those of the 9 compared algorithms on the whole 100-*D* CEC'2017 benchmark suite. c) With regard to "*w/t/l*", FVRADE shows significantly better performance than the 9 compared DE methods on at least 19 problems, while it only displays significant inferiority to them on at most 7 problems. In particular, except for TPDE, FVRADE is better than the other 8 compared algorithms on more than 20 problems. d) With respect to the optimization

Table 3
Summarized comparison results between FVRADE and the four LSHADE variants on the 10-*D*, 30-*D*, 50-*D*, and 100-*D* CEC'2017 benchmark sets with respect to "w/t/l" derived from the Wilcoxon rank sum test and the average rank obtained from the Friedman test.

Problem Set	Index	Problem Property	jSO	FVRA-jSO	FVRADE	LSHADE-SPACMA	FVRA-LSHADE-SPACMA	FVRADE	LSHADE-cnEpSin	FVRA-LSHADE-cnEpSin	FVRADE	EBLSHADE	FVRA-EBLSHADE	FVRADE
CEC'2017–10*D*	*w/t/l*	Unimodal Problems	–	0/2/0	0/2/0	–	0/2/0	0/2/0	–	0/2/0	0/2/0	–	0/2/0	0/2/0
		Simple Multimodal Problems	–	1/4/2	4/2/1	–	1/6/0	4/3/0	–	0/7/0	4/3/0	–	0/6/1	2/5/0
		Hybrid Problems	–	2/8/0	4/3/3	–	0/10/0	2/3/5	–	0/10/0	2/4/4	–	1/9/0	3/6/1
		Composition Problems	–	3/7/0	5/3/2	–	1/9/0	4/5/1	–	2/8/0	6/4/0	–	2/8/0	2/7/1
		Overall	–	6/21/2	13/10/6	–	2/27/0	10/13/6	–	2/27/0	12/13/4	–	3/25/1	7/20/2
	Rank	Overall	**1.67**	2.00	2.33	1.91	**1.84**	2.24	**1.76**	1.93	2.31	**1.69**	2.12	2.19
	CD=0.32	Overall	–	**0.33**	**0.66**	–	0.07	**0.33**	–	0.17	**0.55**	–	**0.43**	**0.50**
CEC'2017–30*D*	*w/t/l*	Unimodal Problems	–	0/1/1	2/0/0	–	0/2/0	2/0/0	–	0/2/0	2/0/0	–	1/1/0	2/0/0
		Simple Multimodal Problems	–	2/5/0	6/0/1	–	0/7/0	6/1/0	–	3/3/1	4/2/1	–	0/7/0	6/1/0
		Hybrid Problems	–	1/7/2	10/0/0	–	0/5/5	7/1/2	–	0/8/2	9/1/0	–	0/10/0	9/0/1
		Composition Problems	–	0/10/0	6/4/0	–	0/9/1	2/5/3	–	1/8/1	2/6/2	–	4/5/1	8/1/1
		Overall	–	3/23/3	24/4/1	–	0/23/6	17/7/5	–	4/21/4	17/9/3	–	5/23/1	25/2/2
	Rank	Overall	1.64	**1.53**	2.83	1.98	**1.67**	2.34	2.00	**1.59**	2.41	**1.45**	1.86	2.69
	CD=0.32	Overall	–	0.10	**1.19**	–	0.31	**0.36**	-	**0.41**	**0.41**	–	**0.41**	**1.24**
CEC'2017–50*D*	*w/t/l*	Unimodal Problems	–	0/0/2	0/2/0	–	0/2/0	2/0/0	–	0/0/2	1/1/0	–	0/0/2	0/1/1
		Simple Multimodal Problems	–	0/6/1	7/0/0	–	0/5/2	7/0/0	–	1/5/1	4/3/0	–	0/3/4	6/0/1
		Hybrid Problems	–	0/4/6	9/1/0	–	0/6/4	10/0/0	–	0/5/5	10/0/0	–	2/8/0	9/1/0
		Composition Problems	–	1/4/5	10/0/0	–	0/9/1	5/5/0	–	0/8/2	4/4/2	–	0/9/1	9/1/0
		Overall	–	1/14/14	26/3/0	–	0/22/7	24/5/0	–	1/18/10	19/8/2	–	2/20/7	24/3/2
	Rank	Overall	2.00	**1.10**	2.90	1.91	**1.33**	2.76	1.93	**1.52**	2.55	1.59	**1.55**	2.86
	CD=0.32	Overall	–	**0.90**	**0.90**	–	**0.59**	**0.84**	–	**0.41**	**0.62**	–	0.03	**1.28**
CEC'2017–100*D*	*w/t/l*	Unimodal Problems	–	0/0/2	0/0/2	–	0/1/1	2/0/0	–	0/0/2	1/0/1	–	0/0/2	0/0/2
		Simple Multimodal Problems	-	0/2/5	6/0/1	-	0/5/2	6/0/1	–	3/3/1	6/0/1	–	1/2/4	6/0/1
		Hybrid Problems	–	1/1/8	8/2/0	–	0/3/7	10/0/0	–	0/4/6	10/0/0	–	1/7/2	9/1/0
		Composition Problems	–	0/5/5	8/2/0	–	0/8/2	9/0/1	–	0/7/3	9/1/0	–	2/5/3	9/1/0
		Overall	–	1/8/20	22/4/3	–	0/17/12	27/0/2	–	3/14/12	26/1/2	–	4/14/11	24/2/3
	Rank	Overall	2.00	**1.24**	2.76	1.88	**1.26**	2.86	1.72	**1.41**	2.86	1.66	**1.59**	2.76
	CD=0.32	Overall	–	**0.76**	**0.76**	–	**0.62**	**0.98**	–	0.31	**1.14**	–	0.07	**1.10**

performance on different categories of optimization problems, on the two unimodal problems (F_1 and F_3), FVRADE shows significantly better performance than the 9 compared algorithms. On the 7 simple multimodal problems (F_4-F_{10}), FVRADE achieves slightly better performance than TPDE and CJADE, but shows much better performance than the other 7 compared methods on at least 5 problems. On the 10 hybrid problems (F_{11}-F_{20}), FVRADE performs slightly better than DiDE and FADE, but significantly outperforms the other 7 compared methods on more than 7 problems. On the 10 composition problems (F_{21}-F_{30}), except for CUSDE, FVRADE shows significant superiority to the other 8 compared DE variants on at least 6 problems, and only presents significant inferiority to them on at most 1 problem. e) Concerning the convergence comparisons shown in Fig. S4, at the first glimpse, FVRADE performs the best among all optimizers concerning the convergence speed and the solution accuracy on 12 problems (F_1, $F_3 \sim F_5$, $F_7 \sim F_8$, F_{13}, F_{16}, F_{21}, F_{24}, F_{26}, and F_{29}). Besides, on F_{12}, F_{15}, F_{18}, F_{19}, and F_{23}, FVRADE shows much better performance than 8 compared methods and displays very similar performance or slightly worse performance than only one compared method. f) Comprehensively speaking, compared with the 9 compared DE variants, FVRADE shows much better performance on the 100-*D* CEC'2017 benchmark suite. In particular, FVRADE also shows much superiority on complicated optimization problems, like the hybrid problems and the composition problems.

All in all, based on the above comparison results, with the dimensionality increasing, FVRADE consistently achieves much better performance than most of the 9 compared methods, and its superiority becomes more and more significant. This verifies that FVRADE is not only effective to solve optimization problems, but also preserves a good scalability to deal with higher-dimensional optimization problems. Besides, FVRADE shows much better performance than most of the 9 compared DE methods particularly on complicated optimization problems, like the multimodal problems, the hybrid problems, and the composition problems in the CEC'2017 benchmark set. This proves that FVRADE has a good capability of tackling complicated optimization problems. The above superiority of FVRADE mainly benefits from "DE/current-to-rwrand/1". Such a mutation strategy offers a chance for each individual to guide the evolution of others and the better one individual is, the higher probability it has to be selected to guide the update of the population. In this way, this mutation strategy could help DE make a good compromise between high diversity and fast convergence to explore the solution space and exploit the found promising regions. With this advantage, FVRADE is able to achieve good performance in coping with optimization problems, especially complicated ones.

4.2.2. Comparisons between FVRADE and the latest LSHADE variants

To further demonstrate the effectiveness of FVRADE, this section conducts experiments to compare FVRADE with the four latest LSHADE variants (jSO [79], LSHADE-cnEpSin [82], LSHADE-SPACMA [77], and EBLSHADE [61]). In particular, the four LSHADE variants and LSHADE [84] all adopt "DE/current-to-pbest/1" developed in JADE [32] as the base mutation operation. To further improve the optimization performance of LSHADE, the four variants developed many other techniques, such as new adaptive parameter strategies for *F* and *CR*, linear adjustment of the population size and local search methods, which make immense contributions to their good optimization performance. However, this paper mainly focuses on developing a novel mutation strategy, named "DE/current-to-rwrand/1", not on the above additional techniques in the four LSHADE variants. Therefore, to further demonstrate the effectiveness of the devised mutation strategy, we replace "DE/current-to-pbest/1" in the four LSHADE variants with "DE/current-to-rwrand/1" to see the optimization performance of these four LSHADE variants. Therefore, four new versions of the four variants are developed, which are denoted as "FVRA-jSO", "FVRA-LSHADE-SPACMA", "FVRA-LSHADE-cnEpSin" and "FVRA-EBLSHADE", respectively.

Subsequently, this paper conducts experiments to compare FVRADE and the four developed new variants with the four LSHADE methods on the 10-*D*, 30-*D*, 50-*D*, and 100-*D* CEC'2017 benchmark sets. Tables S10 ~ S13 in the supplementary material show the detailed comparison results on the CEC'2017 benchmark set with the four settings of the dimensionality, respectively. For better observation, Table 3 summarizes the comparison results between the four LSHADE variants and FVRADE along with its four developed variants on the 10-*D*, 30-*D*, 50-*D*, and 100-*D* CEC'2017 benchmark sets with respect to "*w/t/l*" derived from the Wilcoxon rank sum test and the average rank obtained from the Friedman test. Here, it should be mentioned that in these tables, we compare each LSHADE variant (jSO, LSHADE-SPACMA, LSHADE-cnEpSin, or EBLSHADE) with FVRADE and the associated new variant (FVRA-jSO, FVRA-LSHADE-SPACMA, FVRA-LSHADE-cnEpSin, or FVRA-EBLSHADE). Therefore, "*w/t/l*" in these tables actually indicates that the LSHADE variant performs significantly better than, equivalently to, and significantly worse than FVRADE or the associated new variant, respectively. From these tables, the following conclusions are attained:

1) On the 10-*D* CEC'2017 benchmark suite, a) With respect to the average rank and the critical value of the Nemenyi post hoc test, FVRA-LSHADE-SPACMA and FVRA-LSHADE-cnEpSin achieve highly competitive overall performance with LSHADE-SPACMA and LSHADE-cnEpSin, respectively, while FVRA-jSO and FVRA-EBLSHADE show significantly worse overall performance than jSO and EBLSHADE, respectively on the whole 10-*D* CEC'2017 benchmark suite. b) In view of the Wilcoxon rank sum test results, FVRA-jSO, FVRA-LSHADE-SPACMA, FVRA-LSHADE-cnEpSin, and FVRA-EBLSHADE obtain equivalent performance with jSO, LSHADE-SPACMA, LSHADE-cnEpSin, and EBLSHADE on at least 21 problems. c) The above observations demonstrate that under the frameworks of the four LSHADE variants, "DE/current-to-rwrand/1" performs highly competitively with "DE/current-to-pbest/1" in solving the 10-*D* CEC'2017 benchmark problems.
2) On the 30-*D* CEC'2017 benchmark set, a) The Friedman test results and the critical value of the Nemenyi post hoc test show that FVRA-LSHADE-cnEpSin presents significantly better overall optimization performance than LSHADE-cnEpSin, while FVRA-jSO and FVRA-LSHADE-SPACMA display equivalent overall optimization performance with jSO and LSHADE-SPACMA, respectively. FVRA-EBLSHADE exhibits significantly worse overall optimization performance than EBLSHADE. b) In the view of "*w/t/l*", FVRA-LSHADE-SPACMA is significantly better than LSHADE-SPACMA on 6 problems while it shows no worse performance than LSHADE-SPACMA on any problems. FVRA-jSO, FVRA-LSHADE-cnEpSin, and FVRA-EBLSHADE show no worse performance than jSO, LSHADE-cnEpSin, and EBLSHADE on 26, 25, and 24 problems, respectively. c) These observations substantiate that under the frameworks of LSHADE-SPACMA and LSHADE-cnEpSin, "DE/current-to-rwrand/1" is superior to "DE/current-to-pbest/1", while under the frameworks of jSO and EBLSHADE, "DE/current-to-rwrand/1" is highly competitive to "DE/current-to-pbest/1" in solving the 30-*D* CEC'2017 problems.
3) On the 50-*D* CEC'2017 benchmark suite, a) Concerning the average rank and the critical value of the Nemenyi post hoc test, FVRA-jSO, FVRA-LSHADE-SPACMA, and FVRA-LSHADE-cnEpSin exhibit significantly better overall performance than jSO, LSHADE-SPACMA, and LSHADE-cnEpSin, respectively, while FVRA-EBLSHADE obtains equivalent overall performance with EBLSHADE on the entire 50-*D* CEC'2017 set. b) With respect to the Wilcoxon rank sum test results, FVRA-jSO and FVRA-LSHADE-cnEpSin are significantly superior to jSO and LSHADE-cnEpSin on 14 and 10 problems, respectively, while they are inferior to jSO and LSHADE-cnEpSin both on only 1 problem. FVRA-LSHADE-SPACMA performs significantly better than LSHADE-SPACMA on 7 problems, and shows no inferiority to

Table 4
Summarized comparison results between FVRADE and the 9 state-of-the-art compared algorithms with respect to "*w/t/l*" derived from the Wilcoxon rank sum test and the average rank obtained from the Friedman test on the CEC'2021 suite with the two dimension sizes.

Problem Set	Problem Property	Index	FVRADE	CUSDE	TPDE	CJADE	FADE	DMCDE	DiDE	MRDE	AGDE	JADE
CEC'2021–10*D*	Basic Transformation	*w/t/l*	–	7/1/2	5/2/3	2/3/5	7/1/2	5/0/5	3/3/4	3/1/6	4/0/6	4/3/3
	(000)	*Rank*	5.25	8.20	6.75	**3.15**	6.90	6.00	3.85	4.65	5.05	5.20
		CD=1.21	–	**2.95**	**1.50**	**2.10**	**1.65**	0.75	**1.40**	0.60	0.20	0.05
	Shift Transformation	*w/t/l*	–	6/4/0	5/5/0	2/5/3	5/4/1	5/4/1	1/5/4	1/5/4	6/2/2	3/4/3
	(010)	*Rank*	5.23	6.60	6.45	4.73	6.00	6.05	4.98	**4.45**	5.50	5.03
		CD=1.21	–	**1.37**	**1.22**	0.50	0.77	0.82	0.25	0.78	0.27	0.20
	Shift Rot Transformation	*w/t/l*	–	6/3/1	4/4/2	3/4/3	3/5/2	4/3/3	3/5/2	3/3/4	4/3/3	4/3/3
	(011)	*Rank*	5.33	6.17	6.17	5.20	5.17	5.70	5.73	**4.53**	5.77	5.23
		CD=1.21	–	0.84	0.84	0.13	0.16	0.37	0.40	0.80	0.44	0.10
	Bias Shift	*w/t/l*	–	5/3/2	5/5/0	1/6/3	6/3/1	4/4/2	1/6/3	2/4/4	5/3/2	5/4/1
	Transformation	*Rank*	5.33	5.76	5.93	5.11	5.75	5.68	5.36	**4.93**	5.68	5.49
	(110)	*CD*=1.21	–	0.43	0.60	0.22	0.42	0.35	0.03	0.40	0.35	0.16
	Bias Shift Rot	*w/t/l*	–	7/2/1	5/3/2	3/6/1	5/4/1	4/3/3	4/5/1	3/3/4	5/4/1	5/4/1
	Transformation	*Rank*	5.24	5.82	5.62	5.36	5.28	5.46	5.58	**4.90**	6.40	5.34
	(111)	*CD*=1.21	–	0.58	0.38	0.12	0.04	0.22	0.34	0.34	1.16	0.10
	Overall	*w/t/l*	–	31/13/6	24/19/7	11/24/15	26/17/7	22/14/14	12/24/14	12/16/22	24/12/14	21/18/11
	Overall	*Rank*	4.91	7.58	7.12	4.18	5.81	6.15	5.18	**3.30**	5.76	5.01
	Overall	*CD*=0.73	–	**2.67**	**2.21**	0.73	**0.90**	**1.24**	0.27	**1.61**	**0.85**	0.10
CEC'2021–20*D*	Basic Transformation	*w/t/l*	–	7/1/2	5/2/3	3/5/2	9/1/0	5/1/4	3/3/4	3/3/4	3/3/4	6/3/1
	(000)	*Rank*	4.50	7.95	5.25	4.95	8.75	5.60	5.20	3.75	**3.60**	5.45
		CD=1.21	–	**3.45**	0.75	0.45	**4.25**	1.10	0.70	0.75	0.90	0.95
	Shift Transformation	*w/t/l*	–	7/2/1	6/3/1	2/6/2	9/0/1	6/3/1	5/3/2	5/2/3	3/3/4	4/4/2
	(010)	*Rank*	4.53	6.93	5.98	4.53	7.00	6.43	5.38	4.85	**4.43**	4.98
		CD=1.21	–	**2.40**	**1.45**	0	**2.47**	**1.90**	0.85	0.32	0.10	0.45
	Shift Rot Transformation	*w/t/l*	–	8/2/0	5/4/1	5/4/1	7/1/2	5/4/1	7/3/0	6/4/0	4/5/1	7/3/0
	(011)	*Rank*	**4.62**	6.63	5.38	5.12	5.58	5.58	5.65	5.55	5.37	5.52
		CD=1.21	–	**2.01**	0.76	0.50	0.96	0.96	1.03	0.93	0.75	0.90
	Bias Shift	*w/t/l*	–	8/2/0	6/4/0	1/5/4	8/1/1	8/2/0	5/3/2	3/4/3	4/4/2	5/3/2
	Transformation	*Rank*	5.06	6.39	5.74	**4.86**	6.06	6.04	5.49	4.99	5.01	5.36
	(110)	*CD*=1.21	–	**1.33**	0.68	0.20	1.00	0.98	0.43	0.07	0.05	0.30
	Bias Shift Rot	*w/t/l*	–	7/3/0	5/2/3	5/4/1	7/2/1	5/3/2	7/3/0	4/5/1	4/4/2	7/3/0
	Transformation	*Rank*	**5.04**	6.05	5.57	5.50	5.61	5.52	5.64	5.33	5.28	5.46
	(111)	*CD*=1.21	–	1.01	0.53	0.46	0.57	0.48	0.60	0.29	0.24	0.42
	Overall	*w/t/l*	–	37/10/3	27/15/8	16/24/10	40/5/5	29/13/8	27/15/8	21/18/11	18/19/13	29/16/5
	Overall	*Rank*	**3.57**	8.50	5.83	4.26	7.36	6.39	5.61	4.34	4.00	5.14
	Overall	*CD*=0.73	–	**4.93**	**2.26**	0.69	**3.79**	**2.82**	**2.04**	**0.77**	0.43	**1.57**

LSHADE-SPACMA on any problems. FVRA-EBLSHADE performs significantly better than EBLSHADE on 7 problems, and shows significant inferiority to EBLSHADE on only 2 problems. c) These observations substantiate that under the frameworks of the four LSHADE variants, "DE/current-to-rwrand/1" shows great superiority to "DE/current-to-pbest/1" in solving the 50-*D* CEC'2017 problems.

4) On the 100-*D* CEC'2017 benchmark set, a) In the view of the Friedman test results and the critical value of the Nemenyi post hoc test, FVRA-jSO and FVRA-LSHADE-SPACMA attain significantly better overall performance than jSO and LSHADE-SPACMA, respectively, while FVRA-LSHADE-cnEpSin and FVRA-EBLSHADE achieve equivalent overall performance with LSHADE-cnEpSin and EBLSHADE, respectively on the entire 100-*D* CEC'2017 set. b) In view of the Wilcoxon rank sum test results, FVAR-jSO, FVRA-LSHADE-SPACMA, FVRA-LSHADE-cnEpSin, and FVRA-EBLSHADE perform significantly better than jSO, LSHADE-SPACMA, LSHADE-cnEpSin, and EBLSHADE on 20, 12, 12, and 11 problems, respectively, and show significant inferiority to them on only 1, 0, 3, and 4 problems respectively. c) The above comparison results substantiate that under the frameworks of the four LSHADE variants, "DE/current-to-rwrand/1" shows great superiority to "DE/current-to-pbest/1" in solving the 100-*D* CEC'2017 problems.

All in all, from the above comparison results, we find that with the dimensionality increasing, the four developed new variants of the four LSHADE methods (FVRA-jSO, FVRA-LSHADE-SPACMA, FVRA-LSHADE-cnEpSin, or FVRA-EBLSHADE) show increasingly better performance than the associated original LSHADE variants (jSO, LSHADE-SPACMA, LSHADE-cnEpSin, or EBLSHADE). This reveals that on the one hand, under the frameworks of the four LSHADE variants, "DE/current-to-rwrand/1" shows increasing superiority to the mutation strategy in the four LSHADE variants, namely "DE/current-to-pbest/1"; on the other hand, "DE/current-to-rwrand/1" preserves a good scalability to solve higher-dimensional problems.

4.3. Comparisons on the CEC'2021 benchmark problem set

This section mainly conducts experiments on the latest CEC'2021 benchmark set [50] with the two settings of the dimensionality to further verify the effectiveness of FVRADE. Though there are totally 50 optimization problems in the CEC'2021 set, the 10 basic problems are mainly selected from the CEC'2017 benchmark set [49]. Therefore, this paper directly adopts the population size settings of all methods fine-tuned on the CEC'2017 benchmark set as shown in Table 1 to optimize the problems in the CEC'2021 set. In particular, for the 10-*D* CEC'2021 problems, we utilize the fine-tuned population sizes shown in Table 1 on the 10-*D* CEC'2017 problems. For the 20-*D* CEC'2021 problems, we employ the fine-tuned population sizes shown in Table 1 on the 30-*D* CEC'2017 problems. In addition, according to [50], the maximum number of fitness evaluations is set as 2.0E+05 and 1.0E+06 for all algorithms to solve the 10-*D* and the 20-*D* CEC'2021 benchmark problems, respectively.

Table S14 and Table S15 in the supplementary material present the detailed comparisons between FVRADE and the 9 compared DE variants on the 10-*D* and the 20-*D* CEC'2021 benchmark suites, respectively. For better comparisons, Table 4 shows the summarized comparison results

between FVRADE and the 9 state-of-the-art compared algorithms with respect to "*w/t/l*" derived from the Wilcoxon rank sum test and the average rank obtained from the Friedman test on the CEC'2021 suite with the two dimension sizes.

From Table 4, Table S14, and Table S15, we get the following conclusions from the comparison results between FVRADE and the 9 compared algorithms:

1) On the 10-*D* CEC'2021 problems, a) With respect to the basic transformation of the 10 problems, the Friedman test results along with the Nemenyi post hoc test show that FVRADE performs significantly better than CUSDE, TPDE, and FADE, obtains equivalent performance with DMCDE, MRDE, AGDE, and JADE, and shows inferiority to CJADE, and DiDE. In terms of "*w/t/l*", FVRADE shows significantly better performance than CUSDE and FADE both on 7 problems and displays inferiority to them both on only 2 problems. In comparison with TPDE and JADE, FVRADE performs no worse than them on 7 problems. In competition with CJADE, DiDE, MRDE, and AGDE, FVRADE performs slightly worse than them. Compared with DMCDE, FVRADE achieves equivalent performance. b) In view of the shift transformation of the 10 problems, the Friedman test results and the Nemenyi post hoc test show that FVRADE shows significant overall superiority to CUSDE, and TPDE, while it presents equivalent overall performance with the other 7 compared methods. Concerning "*w/t/l*", FVRADE presents significant superiority to CUSDE, TPDE, FADE, DMCDE, and AGDE on more than 5 problems, and only shows inferiority to them on at most 2 problems. Besides, FVRADE performs equivalently to CJADE and JADE, and shows slightly inferiority to DiDE and MRDE. c) Regarding the shift rot transformation of the 10 problems, the Friedman test results along with the Nemenyi post hoc test show that FVRADE obtains equivalent overall performance with all 9 compared methods. In terms of "*w/t/l*", FVRADE is significantly better than CUSDE on 6 problems and displays inferiority to it on only 1 problem. Compared with MRDE, FVRADE is significantly better on 3 problems, but shows worse performance on 4 problems. In comparison with the other 7 compared methods, FVRADE presents no worse performance on at least 7 problems. d) Concerning the bias shift transformation of the 10 problems, the Friedman test results and the Nemenyi post hoc test show that FVRADE still obtains equivalent overall performance with the 9 compared methods. Regarding "*w/t/l*", FVRADE shows significantly better performance than CUSDE, TPDE, FADE, AGDE, and JADE on at least 5 problems and presents significant inferiority to them on at most 2 problems. As compared with the other four DE variants (CJADE, DMCDE, DiDE, and MRDE), FVRADE presents no worse performance than them on at least 6 problems. e) As for the bias shift rot transformation of the 10 problems, the Friedman test results together with the Nemenyi post hoc test show that FVRADE still attains equivalent overall performance with the 9 compared methods. With regard to "*w/t/l*", FVRADE shows significantly better performance than CUSDE, TPDE, FADE, AGDE, and JADE on more than 5 problems, and presents inferiority to them on at most 2 problems. In competition with the other four DE variants (CJADE, DMCDE, DiDE, and MRDE), FVRADE shows no worse performance than then on at least 6 problems. f) From the perspective of the whole 50 optimization problems, with respect to the Friedman test results and the Nemenyi post hoc test, FVRADE shows significantly better overall optimization performance than 5 compared methods (CUSDE, TPDE, FADE, DMCDE, and AGDE), displays competitive overall performance with 3 compared methods (CJADE, DiDE, and JADE), and only presents significant inferiority to MRDE.
2) On the 20-*D* CEC'2021 problems, a) With respect to the basic transformation of the 10 problems, the Friedman test results along with the Nemenyi post hoc test show that FVRADE performs significantly better than CUSDE and FADE, and obtains equivalent overall performance with the other 7 compared methods. Regarding "*w/t/l*", FVRADE attains much better performance than CUSDE, TPDE, FADE, DMCDE, and JADE on more than 5 problems and only performs worse than them on at most 4 problems. In comparison with the other 4 DE algorithms, FVRADE presents no worse performance on at least 6 problems. b) In view of the shift transformation of the 10 problems, the Friedman test results together with the Nemenyi post hoc test show that FVRADE shows significant overall superiority to CUSDE, TPDE, FADE, and DMCDE, and shows highly competitive overall performance with the other 5 compared methods. Concerning "*w/t/l*", FVRADE is significantly better than CUSDE, TPDE, FADE, DMCDE, DiDE, and MRDE on more than 5 problems, and only worse than them on at most 3 problems. Competed with the other 3 compared variants, FVRADE exhibits no worse performance on at least 6 problems. c) Regarding the shift rot transformation of the 10 problems, the Friedman test results along with the Nemenyi post hoc test show that FVRADE shows much better overall performance than CUSDE and obtains highly competitive overall performance with the other 8 compared DE methods. With respect to "*w/t/l*", except for AGDE, FVRADE is significantly better than the other 8 compared methods on at least 5 problems and only exhibits inferiority to them on at most 2 problems. Even compared with AGDE, FVRADE shows no worse performance on 9 problems. d) Concerning the bias shift transformation of the 10 problems, the Friedman test results and the Nemenyi post hoc test show that FVRADE attains significantly better overall performance than CUSDE and achieves equivalent overall performance with the other 8 compared methods. Regarding "*w/t/l*", FVRADE performs significantly better than CUSDE, TPDE, FADE, DMCDE, DiDE, and JADE on at least 5 problems, and only shows inferiority to them on at most 2 problems. In comparison with the other 3 compared algorithms, FVRADE displays no worse performance on at least 6 problems. e) As for the bias shift rot transformation of the 10 problems, the Friedman test results along with the Nemenyi post hoc test show that FVRADE obtains equivalent overall performance with the 9 compared methods. From the perspective of "*w/t/l*", except for MRDE and AGDE, FVRADE performs significantly better than the other 7 compared methods on at least 5 problems and only displays inferiority to them on at most 3 problems. Even compared with MRDE and AGDE, FVRADE gains no worse performance on at least 8 problems. f) From the perspective of the whole 50 optimization problems, in view of the Friedman test results and the Nemenyi post hoc test, FVRADE presents significantly better overall optimization performance than 7 compared methods (CUSDE, TPDE, FADE, DMCDE, DiDE, MRDE, and JADE), and displays competitive overall performance with 2 compared methods (CJADE and AGDE).

In summary, from the above comparison results, it is found that the superiority of FVRADE to the 9 compared methods is more significant on the 20-*D* CEC'2021 problems than on the 10-*D* CEC'2021 problems. On the one hand, it is proven that FVRADE is effective to solve the latest CEC'2021 benchmark optimization problems; on the other hand, it is also verified that FVRADE preserves a good scalability to solve higher dimensional problems.

4.4. Deep investigation on FVRADE

In this section, this paper mainly conducts in-depth observations on FVRADE to investigate the effectiveness of its two main components, namely the devised "DE/current-to-rwrand/1" and the additional archive used in the mutation strategy.

4.4.1. Effectiveness of the devised mutation strategy

First, investigation experiments are conducted on the 100-*D* CEC'2017 benchmark problem set to verify the effectiveness of "DE/current-to-rwrand/1". Since "DE/current-to-best/1" and "DE/current-to-rand/1" are two special cases of "DE/current-to-rwrand/1", this

paper first replaces "DE/current-to-rwrand/1" in FVRADE with the two mutation strategies, developing its two variants. In addition, "DE/current-to-pbest/1" is also similar to "DE/current-to-rwrand/1". Therefore, this paper also develops a variant of FVRADE by replacing the mutation scheme with "DE/current-to-pbest/1". In addition, compared with the above three classical mutation strategies, "DE/current-to-rwrand/1" made an improvement on the random difference vector, namely utilizing the better one between the two random individuals as the first exemplar while adopting the worse one as the second exemplar when generating the random difference vector. To verify the effectiveness of this improvement, we remove this improvement and thus develop another variant of "DE/current-to-rwrand/1", which is named "DE/current-to-rwrand/1-WD".

Additionally, in "DE/current-to-rwrand/1", the selection probability of each individual is calculated based on its function value ranking as shown in Eq.(12) and Eq.(11). To demonstrate the effectiveness of this way of probability calculation, this paper first denotes this FVRADE as "FVRADE-REW" and then develops another two ways of probability calculation to make comparisons. The first is to directly adopt the function value ranking to compute the selection probability of each individual as follows:

$$p_i = \frac{NP - rank(i) + 1}{\sum_{j=1}^{NP} j} \tag{14}$$

where NP is the size of the population, $rank(i)$ is the function value ranking of the ith individual after all individuals are sorted in the ascending order of the function values. By replacing the selection probability calculation in FVRADE as shown in Eq.(11) with Eq.(14), a new variant of FVRADE is developed, which is named "FVRADE-RW".

Another way of calculating the selection probability is to directly use the function values of individuals in the following way:

$$p_i = \frac{f'(x_i)}{\sum_{j=1}^{NP} f'(x_j)} \tag{15}$$

where $f'(x_i)$ is the normalized function value of the ith individual, which is computed as follows:

$$f'(x_i) = \frac{f_{\max} - f(x_i) + \xi}{f_{\max} - f_{\min} + \xi} \tag{16}$$

where $f_{\max}$ and $f_{\min}$ are the largest function value and the smallest function value in the current population, respectively. ξ is a small real value used to avoid the zero denominator. By replacing the selection probability calculation in FVRADE as shown in Eq.(11) with Eq.(15), another new variant of FVRADE is developed, which is named "FVRADE-FW".

After the above preparations, this paper conducts experiments on the 100-D CEC'2017 benchmark problems to compare FVRADE with the above resultant variants. Table S16 in the supplementary material shows the comparison results. From this table, the following discoveries are obtained:

1) As shown in the left part of Table S16, from the comparisons among the five mutation strategies, namely "DE/current-to-rwrand/1", "DE/current-to-best/1", "DE/current-to-rand/1", "DE/current-to-pbest/1", and "DE/current-to-rwrand/1-WD", we find that a) "DE/current-to-rwrand/1" achieves the lowest rank regarding the Friedman test results. This means that "DE/current-to-rwrand/1" helps FVRADE perform the best on the whole 100-D CEC'2017 benchmark set. b) According to the critical value of the Nemenyi post hoc test, the overall performance of "DE/current-to-rwrand/1" on the entire 100-D CEC'2017 set is significantly better than the four compared mutation strategies. c) According to the Wilcoxon rank sum test results, "DE/current-to-rwrand/1" is significantly superior to the four compared mutation strategies on at least 11 problems and inferior to them on at most 2 problems. d) The superiority of "DE/current-to-rwrand/1" to "DE/current-to-best/1", "DE/current-to-rand/1", and "DE/current-to-pbest/1" demonstrates the effectiveness of the selection strategy for the guiding exemplars of individuals based on the devised non-linear selection probability computation method. The dominance of "DE/current-to-rwrand/1" to "DE/current-to-rwrand/1-WD" verifies the effectiveness of the directional random difference vector. Together, the effectiveness of "DE/current-to-rwrand/1" is substantiated.
2) As displayed in the right part of Table S16, with respect to the comparisons among the three ways of probability calculation, FVRADE-REW obtains the lowest rank among all the three variants of FVRADE. This proves that FVRADE-REW performs the best on the whole 100-D CEC'2017 benchmark set. In terms of the critical value of the Nemenyi post hoc test, FVRADE-REW presents significantly better overall performance than the two compared variants. Furthermore, the Wilcoxon rank sum test results prove that FVRADE-REW significantly outperforms the other two variants on 22 and 23 problems, respectively. These observations verify the effectiveness of the way of the selection probability calculation in FVRADE. Such a way of probability calculation is superior because it decreases the probability exponentially as the ranking increases. In this way, better individuals have exponentially higher chances, while worse individuals preserve exponentially lower chances to act as the guiding exemplars to direct the mutation of the population. As a result, the difference between individuals is enlarged, and thus "DE/current-to-rwrand/1" expectedly affords directional but diverse mutations for the population, which is profitable for FVRADE to compromise high diversity and fast convergence well during the evolution.

As a whole, the above comparison results have verified that "DE/current-to-rwrand/1" is very effective to aid FVRADE to obtain good optimization performance in coping with optimization problems. Such effectiveness mainly profits from the proposed selection probability calculation, which lets better individuals have exponentially larger selection probabilities, while worse individuals preserve exponentially smaller probabilities to be selected to guide the mutation of others. With this selection probability computation method, FVRADE is expected to explore the solution space and exploit the found promising regions appropriately to seek satisfactory solutions to optimization problems.

4.4.2. Effectiveness of the additional archive

Subsequently, this paper verifies the validity of the external archive used in "DE/current-to-rwrand/1" for generating random difference vectors. To this end, this paper first removes the archive from FVRADE, developing a new variant of FVRADE, which is named "FVRADE-WA". Then, experiments are conducted on the 100-D CEC'2017 benchmark set to compare FVRADE with FVRADE -WA. Table S17 in the supplementary material shows the comparison results between these two variants.

From Table S17, FVRADE achieves a much lower rank value than FVRADE-WA. This demonstrates that FVRADE with the additional archive obtains the best overall performance on the whole 100-D CEC'2017 set. Besides, with respect to the critical value of the Nemenyi post hoc test, FVRADE shows significantly better overall performance than FRADE-WA. In addition, FVRADE significantly outperforms FVRADE-WA on 17 benchmark problems, while FVRADE-WA significantly outperforms FVRADE on only 2 problems. This further verifies that FVRADE with the archive performs much better.

Comprehensively, the above experiments have demonstrated that the additional archive is of great help to aid FVRADE to achieve good performance. This is because the additional archive affords more candidates to generate the random difference vectors, and thus more diversified mutation vectors can be generated. It is very profitable for FVRADE to keep high search diversity to avoid falling into local regions

Table 5
The optimal settings of the population sizes (*NP*) for all methods on the CEC'2011 benchmark set.

Algorithm	FVRADE	CUSDE (2022)	TPDE (2021)	CJADE (2021)	FADE (2020)	DMCDE (2020)	DiDE (2020)	MRDE (2019)	AGDE (2019)	JADE (2009)
NP	60	90	60	60	3 × 25	150	90	3 × 20	30	60

Table 6
Comparison results between FVRADE and the 9 compared state-of-the-art DE methods on the six CEC'2011 real-world optimization problems.

F	Quality	FVRADE	CUSDE	TPDE	CJADE	FADE	DMCDE	DiDE	MRDE	AGDE	JADE
F_1	Mean	1.47E+00	4.87E+00	2.63E+00	2.91E+00	2.71E+00	2.06E+00	2.16E+00	1.28E+00	5.59E+00	7.16E+00
	Std	3.73E+00	6.33E+00	4.96E+00	4.41E+00	4.79E+00	4.18E+00	4.76E+00	2.47E+00	4.62E+00	6.38E+00
	p-value	–	**1.54E-02**$^{+}$	**1.75E-11**$^{+}$	**7.21E-11**$^{+}$	**1.38E-08**$^{+}$	2.27E-01$^{=}$	**1.80E-02**$^{+}$	4.53E-10^{-}	**2.54E-12**$^{+}$	**2.01E-14**$^{+}$
F_2	Mean	−2.16E+01	−2.12E+01	−1.26E+01	−2.04E+01	−1.18E+01	−1.71E+01	−2.47E+01	−1.19E+01	−1.61E+01	−2.07E+01
	Std	1.53E+00	3.76E+00	1.60E+00	1.02E+00	1.04E+00	4.90E+00	2.60E+00	1.35E+00	1.25E+00	1.24E+00
	p-value	–	5.51E-01$^{=}$	**3.30E-18**$^{+}$	**4.55E-06**$^{+}$	**3.30E-18**$^{+}$	**1.89E-06**$^{+}$	4.64E-10^{-}	**3.30E-18**$^{+}$	**6.68E-18**$^{+}$	**9.46E-04**$^{+}$
F_4	Mean	1.57E+01	2.03E+01	1.85E+01	1.68E+01	1.51E+01	1.76E+01	1.44E+01	1.60E+01	1.84E+01	1.57E+01
	Std	2.81E+00	1.69E+00	3.31E+00	3.26E+00	2.36E+00	3.61E+00	9.55E-01	2.88E+00	3.30E+00	2.87E+00
	p-value	–	**2.75E-12**$^{+}$	**3.53E-04**$^{+}$	**3.68E-02**$^{+}$	3.33E-01$^{=}$	4.16E-01$^{=}$	3.24E-01$^{=}$	1.68E-01$^{=}$	**2.57E-06**$^{+}$	4.96E-01$^{=}$
F_7	Mean	1.31E+00	1.29E+00	1.27E+00	1.30E+00	1.50E+00	1.62E+00	7.34E-01	1.45E+00	1.43E+00	1.25E+00
	Std	1.60E-01	2.99E-01	1.02E-01	8.87E-02	1.14E-01	2.09E-01	1.47E-01	1.20E-01	1.06E-01	1.07E-01
	p-value	–	4.62E-01$^{=}$	5.04E-03^{-}	9.83E-02$^{=}$	**9.94E-10**$^{+}$	**3.39E-12**$^{+}$	5.61E-17^{-}	**1.40E-06**$^{+}$	**1.90E-05**$^{+}$	1.51E-03^{-}
F_{12}	Mean	1.57E+01	1.59E+01	1.92E+01	1.86E+01	2.15E+01	1.62E+01	2.25E+01	2.13E+01	1.98E+01	1.93E+01
	Std	1.71E+00	7.15E-01	2.01E+00	1.86E+00	1.64E+00	1.36E+00	6.99E+00	2.02E+00	2.04E+00	2.35E+00
	p-value	–	9.04E-02$^{=}$	**1.81E-13**$^{+}$	**6.20E-11**$^{+}$	**3.01E-17**$^{+}$	**2.67E-02**$^{+}$	**1.66E-07**$^{+}$	**1.84E-16**$^{+}$	**2.48E-14**$^{+}$	**8.70E-12**$^{+}$
F_{13}	Mean	1.73E+01	1.70E+01	1.67E+01	2.00E+01	1.78E+01	1.59E+01	1.51E+01	1.74E+01	2.06E+01	2.15E+01
	Std	3.26E+00	3.20E+00	3.59E+00	3.16E+00	4.07E+00	2.72E+00	3.04E+00	3.34E+00	2.53E+00	3.07E+00
	p-value	–	3.00E-01$^{=}$	3.15E-01$^{=}$	**6.10E-05**$^{+}$	8.41E-01$^{=}$	2.84E-03^{-}	1.65E-04^{-}	7.18E-01$^{=}$	**3.52E-07**$^{+}$	**1.78E-09**$^{+}$
Overall	*w/t/l*	–	2/4/0	4/1/1	5/1/0	4/2/0	3/2/1	2/1/3	3/2/1	6/0/0	4/1/1
	Rank	3.17	5.17	5.50	5.83	7.17	5.17	**3.00**	6.17	7.83	6.00
	CD=1.42	–	**2.00**	**2.33**	**2.67**	**4.00**	**2.00**	0.17	**3.00**	**4.67**	**2.83**

Algorithm 1
The pseudocode of DE (DE/current-to-rand/1).

Begin: Population size *NP*, Dimension size *D*, and Maximum fitness evaluations FES_{max};

Randomly initialize *NP* individuals and calculate their function values;//
initialization
$fes = NP$;
While ($fes \le FES_{max}$) **do** //**main iteration**
For $i = 1{:}NP$ **do**
Randomly choose three distinct individuals $\boldsymbol{x}_{rand1}$, $\boldsymbol{x}_{rand2}$ and $\boldsymbol{x}_{rand3}$ from the population;
$v_{i,t} = \boldsymbol{x}_{i,t} + F \times (\boldsymbol{x}_{rand1} - \boldsymbol{x}_{i,t}) + F \times (\boldsymbol{x}_{rand2} - \boldsymbol{x}_{rand3})$ //**mutation**
$d_{rand} = \mathrm{randint}(1, D)$; //randomly select a dimension from all dimensions
For $d = 1{:}D$ **do** //**crossover**
If rand(0,1)$\le CR$ or $d == d_{rand}$ **then**
$u_{i,t}^{d} = v_{i,t}^{d}$;
Else
$u_{i,t}^{d} = x_{i,t}^{d}$;
End If
End For
Compute the function value of the offspring $\boldsymbol{u}_i$;
$fes = fes + 1$;
If $f(\boldsymbol{u}_{i,t}) < f(\boldsymbol{x}_{i,t})$ **then** //**selection**
$\boldsymbol{x}_{i,t+1} = \boldsymbol{u}_{i,t}$;
End If
End For
End While
Get the best individual $\boldsymbol{x}_{gbest}$ in the population and its function value $f(\boldsymbol{x}_{gbest})$;

End: Output $\boldsymbol{x}_{gbest}$ and $f(\boldsymbol{x}_{gbest})$

during the evolution.

4.5. Comparisons on real-world optimization problems

At last, in this section, we conduct experiments to verify the effectiveness of FVRADE in solving real-world optimization problems. To this end, we choose 6 real-world optimization problems, where no constraints are involved or only bound constraints are involved, from the CEC'2011 benchmark real-world optimization problem set [51], to make comparisons between FVRADE and the selected 9 compared state-of-the-art and representative DE variants. Specifically, the selected six problems are Parameter Estimation for Frequency-Modulated (FM) Sound Waves (F_1), Lennard-Jones Potential Problem (F_2), Optimal Control of a Non-Linear Stirred Tank Reactor (F_4), Spread Spectrum Radar Polly phase Code Design (F_7), Messenger: Spacecraft Trajectory Optimization Problem (F_{12}) and Cassini 2: Spacecraft Trajectory Optimization Problem (F_{13}). Following the guideline in [51], the number of function evaluations is set as 50,000 for all algorithms.

To make fair comparisons, we first fine-tune the population size for all methods. Table S18 in the supplementary material shows the fine-tuning results of all algorithms with different settings of the population size. Based on the average rank obtained from the Friedman test, we get the optimal settings of the population size for all methods, which are summarized in Table 5. It should be mentioned that we do not fine-tune the population size of FADE and MRDE, because they adopt an adaptive strategy to dynamically adjust the population size during the evolution. Based on the fine-tuned population sizes, we compare FVRADE with the 9 compared DE variants on the six real-world optimization problems. Table 6 shows their optimization results and their comparisons on the six CEC'2011 real-world optimization problems.

From Table 6, the following findings can be found: 1) In view of the Friedman test results, FVRADE achieves the second place among all methods. Its rank value is slightly larger than that of DiDE, but is much smaller than those of the other 8 compared methods. 2) From the perspective of the critical value of the Nemenyi post hoc test, FVRADE shows equivalent overall performance with DiDE, but presents significantly better overall performance than the other 8 compared methods. 3) According to the Wilcoxon rank sum test results, FVRADE shows significantly better performance than TPDE, CJADE, FADE, AGDE, and JADE on more than 4 problems, and displays significant inferiority to them on at most 1 problem. Compared with the other 4 DE methods, FVRADE obtains highly competitive or slightly better performance than them.

Overall, the above comparison results prove that FVRADE is capable

Algorithm 2
The pseudocode of FVRADE.

Begin: Population size NP, Dimension size D, Maximum fitness evaluations FES_{max};

Randomly create NP individuals and calculate their function values; $fes = NP$;
//initialization
μ_{CR}=0.5; μ_F=0.5; $\boldsymbol{A} = \varnothing$;
Calculate the selection probabilities according to Eqs. (11) and (12);
While ($fes \leq FES_{max}$) **do //main iteration**
$\boldsymbol{S}_F = \varnothing$; $\boldsymbol{S}_{CR} = \varnothing$;
Generate a set of $\boldsymbol{CR}=\{CR_i | i = 1, 2, \ldots, NP\}$, and a set of $\boldsymbol{F}=\{F_i | i = 1, 2, \ldots, NP\}$
according to Eq. (9);
Sort $\boldsymbol{CR}$ from the smallest to the largest;
Sort all individuals from the best to the worst;
For $i = 1{:}NP$ **do**
Obtain the associated CR_i according to the ranking of $\boldsymbol{x}_{i,t}$;
Randomly select $\boldsymbol{x}_{i,rwrand,t,}$ by the roulette selection strategy based on the
calculated selection probabilities;
Randomly choose one individual from $\boldsymbol{P}$ and another individual from $\boldsymbol{P} \cup \boldsymbol{A}$:
$\boldsymbol{x}_{rand1,t}$ and $\boldsymbol{x}_{rand2,t}$, $\boldsymbol{x}_{rand1,t} \neq \boldsymbol{x}_{rand2,t} \neq \boldsymbol{x}_{i,t} \neq \boldsymbol{x}_{i,rwrand,t}$;
If $f(\boldsymbol{x}_{rand2,t}) < f(\boldsymbol{x}_{rand1,t})$ **then**
Swap $\boldsymbol{x}_{rand1,t}$ and $\boldsymbol{x}_{rand2,t}$;
End If
Generate the mutation vector $\boldsymbol{v}_{i,t}$ based on Eq. (13); **/mutation**
Randomly select a dimension: d_{rand} = randint(1, D);
For $d = 1{:}D$ **do//crossover**
If rand(0,1)$\leq CR_i$ or $d == d_{rand}$ **then**
$u_{i,t}^d = v_{i,t}^d$;
Else
$u_{i,t}^d = x_{i,t}^d$;
End If
End For
End For
For $i = 1{:}NP$ **do //selection**
If $f(\boldsymbol{u}_{i,t}) < f(\boldsymbol{x}_{i,t})$ **then**
If $\boldsymbol{A}$ is not full **then**
Add $\boldsymbol{x}_{i,t}$ into the archive $\boldsymbol{A}$;
Else
Choose an individual randomly from $\boldsymbol{A}$ and the better one between this
individual and $\boldsymbol{x}_{i,t}$ is retained in $\boldsymbol{A}$;
End If
$\boldsymbol{x}_{i,t+1} = \boldsymbol{u}_{i,t}$; $\boldsymbol{S}_{CR} \leftarrow CR_i$, $\boldsymbol{S}_F \leftarrow F_i$;
End If
End For
Update μ_{CR} and μ_F based on Eq. (10), and $fes = fes + NP$;
End While
Get the best individual $\boldsymbol{x}_{gbest}$ and its function value $f(\boldsymbol{x}_{gbest})$;
End Output $\boldsymbol{x}_{gbest}$ and $f(\boldsymbol{x}_{gbest})$

of solving real-world optimization problems.

5. Conclusion

This paper has devised a simple yet effective mutation mechanism called "DE/current-to-rwrand/1" to help DE effectively solve optimization problems. Specifically, in this mutation strategy, an exponential weight function was devised to calculate the selection probability of each individual based on its function value ranking after all individuals are sorted from the best to the worst. In this way, better individuals preserve exponentially larger selection probabilities, while worse individuals have exponentially smaller probabilities to be used as the guiding exemplars to direct the mutation of others. With this mechanism, "DE/current-to-rwrand/1" is expected to generate directional but diverse mutation vectors for DE. In particular, "DE/current-to-rwrand/1" is a general mutation framework. With different settings of one parameter in the weight calculation, "DE/current-to-rwrand/1" preserves different power in exploring and exploiting the solution space. Particularly, "DE/current-to-rand/1" and "DE/current-to-best/1" are two special cases of "DE/current-to-rwrand/1" with the parameter in the exponential weight function set as an infinite value and an infinitesimal value, respectively. Embedded "DE/current-to-rwrand/1" and the adaptive parameter strategy for F and CR in JADE_Sort into the DE framework, a new DE, named function value ranking aware DE (FVRADE) was developed.

Experiments have been carried out on the widely used CEC'2017 benchmark problem set along with four different settings of the dimensionality (namely 10-D, 30-D, 50-D and 100-D) and the latest CEC'2021 benchmark set with two different settings of the dimensionality (namely 10-D and 20-D) to substantiate the effectiveness of FVRADE. Compared with 9 popular and state-of-the-art DE methods, FVRADE consistently attains highly competitive performance or even much better performance on the 10-D, 30-D, 50-D and 100-D CEC'2017 benchmark sets and the 10-D and 20-D CEC'2021 benchmark suites. Therefore, it is concluded that FVRADE not only is effective to solve various optimization problems, but also preserves a good scalability to deal with optimization problems with different dimensionalities. Besides, by embedding "DE/current-to-rwrand/1" into four latest LSHADE variants, its effectiveness is further demonstrated. In addition, in-depth investigations on FVRADE have further proved the effectiveness of "DE/current-to-rwrand/1". At last, comparison results between FVRADE and the 9 compared latest DE methods on six CEC'2011 real-world optimization problems demonstrate that FVRADE is also effective in solving real-world optimization problems.

In FVRADE, this paper keeps the parameter σ in the selection probability calculation fixed, which needs effort to fine-tune. Therefore, in the future, we mainly concentrate on devising an adaptive strategy to dynamically adjust the setting of σ to further boost the optimization performance of FVRADE and at the same time to eliminate the parameter sensitivity of FVRADE. Another future direction is to employ FVRADE to solve real-world application problems.

Credit author statement

Author Contributions: Dong Liu: Conceptualization, supervision, methodology, and formal analysis. **Hao He:** Methodology, software, validation, and writing—original draft preparation. **Qiang Yang:** Methodology, supervision, formal analysis, and writing—original draft preparation. **Yi-Qiao Wang:** Formal analysis, and writing—review and editing. **Sang-Woon Jeon:** Formal analysis and writing—review and editing. **Jun Zhang:** Conceptualization, methodology, and writing—review and editing. All authors have read and agreed to the submitted version of the manuscript.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgement

This work was supported in part by the National Natural Science Foundation of China under Grant 62006124, and 62072160, in part by the Natural Science Foundation of Jiangsu Province under Project BK20200811, and in part by the National Research Foundation of Korea under Grant NRF-2021H1D3A2A01082705.

Supplementary materials

Supplementary material associated with this article can be found, in the online version, at doi:10.1016/j.swevo.2023.101282.

References

[1] W. Yi, Y. Chen, Z. Pei, J. Lu, Adaptive differential evolution with ensembling operators for continuous optimization problems, Swarm Evol. Comput. 69 (2022), 100994.
[2] Q. Yang, J.-.Q. Yan, X.-.D. Gao, D.-.D. Xu, Z.-.Y. Lu, J. Zhang, Random neighbor elite guided differential evolution for global numerical optimization, Inf. Sci. (Ny) 607 (2022) 1408–1438.
[3] R. Storn, K. Price, Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces, J. Global Optim. 11 (4) (1997) 341–359.
[4] S. Das, A. Abraham, U.K. Chakraborty, A. Konar, Differential evolution using a neighborhood-based mutation operator, IEEE Trans. Evol. Comput. 13 (3) (2009) 526–553.
[5] W. Deng, J. Xu, Y. Song, H. Zhao, Differential evolution algorithm with wavelet basis function and optimal mutation strategy for complex optimization problem, Appl. Soft. Comput. 100 (2021), 106724.
[6] Q. Yang, H.Y. Xie, W.N. Chen, J. Zhang, Multiple parents guided differential evolution for large scale optimization, IEEE Congress Evolut. Comput. (2016) 3549–3556.
[7] X. Zhou, G. Zhang, Differential evolution with underestimation-based multimutation strategy, IEEE Trans. Cybern. 49 (4) (2019) 1353–1364.
[8] Z. Tan, Y. Tang, K. Li, H. Huang, S. Luo, Differential evolution with hybrid parameters and mutation strategies based on reinforcement learning, Swarm Evol. Comput. 75 (2022), 101194.
[9] X. Liu, et al., Historical and heuristic-based adaptive differential evolution, IEEE Trans. Syst. Man. Cybern. Syst. 49 (12) (2019) 2623–2635.
[10] W.-.J. Yu, J.-.Y. Ji, Y.-.J. Gong, Q. Yang, J. Zhang, A tri-objective differential evolution approach for multimodal optimization, Inf. Sci. (Ny) 423 (2018) 1–23.
[11] J. Liang, et al., A clustering-based differential evolution algorithm for solving multimodal multi-objective optimization problems, Swarm Evol. Comput. 60 (2021), 100788.
[12] X. Cai, L. Gao, X. Li, H. Qiu, Surrogate-guided differential evolution algorithm for high dimensional expensive problems, Swarm Evol. Comput. 48 (2019) 288–311.
[13] F.F. Wei, et al., A classifier-assisted level-based learning swarm optimizer for expensive optimization, IEEE Trans. Evol. Comput. 25 (2) (2021) 219–233.
[14] X.G. Zhou, C.X. Peng, J. Liu, Y. Zhang, G.J. Zhang, Underestimation-assisted global-local cooperative differential evolution and the application to protein structure prediction, IEEE Trans. Evol. Comput. 24 (3) (2020) 536–550.
[15] M. Rakshit, S. Bhattacharjee, G. Garai, A. Chakrabarti, A novel differential evolution algorithm for tone reservation based peak to average power ratio reduction technique in orthogonal frequency division multiplexing systems, Swarm Evol. Comput. 72 (2022), 101086.
[16] S. Cheng, B. Liu, Y. Shi, Y. Jin, B. Li, Evolutionary computation and big data: key challenges and future directions. *Data Mining and Big Data*, Cham, Springer International Publishing, 2016, pp. 3–14.
[17] Z. Lu, S. Liang, Q. Yang, B. Du, Evolving block-based convolutional neural network for hyperspectral image classification, IEEE Trans. Geosci. Remote Sens. 60 (2022) 1–21.
[18] Q. Yang, W.N. Chen, T. Gu, H. Jin, W. Mao, J. Zhang, An adaptive stochastic dominant learning swarm optimizer for high-dimensional optimization, IEEE Trans. Cybern. 52 (3) (2022) 1960–1976.
[19] Y. Duan, J.S. Edwards, Y.K. Dwivedi, Artificial intelligence for decision making in the era of big data – evolution, challenges and research agenda, Int. J. Inf. Manage 48 (2019) 63–71.
[20] J. Bateman, J. Wildfeuer, T. Hiippala, Multimodality: foundations, research and analysis – a problem-oriented introduction, Walter de Gruyter GmbH & Co KG (2017).
[21] Q. Yang, W.N. Chen, Y. Li, C.L.P. Chen, X.M. Xu, J. Zhang, Multimodal estimation of distribution algorithms, IEEE Trans. Cybern. 47 (3) (2017) 636–650.
[22] M.F. Ahmad, N.A.M. Isa, W.H. Lim, K.M. Ang, Differential evolution: a recent review based on state-of-the-art works, Alexandria Eng. J. 61 (5) (2022) 3831–3872.
[23] X. Xia, et al., A fitness-based adaptive differential evolution algorithm, Inf. Sci. (Ny) 549 (2021) 116–141.
[24] K.R. Opara, J. Arabas, Differential Evolution: a Survey of Theoretical Analyses, Swarm Evol. Comput. 44 (2019) 546–558.
[25] A.W. Mohamed, A.A. Hadi, A.K. Mohamed, Differential evolution mutations: taxonomy, comparison and convergence analysis, IEEE Access 9 (2021) 68629–68662.
[26] R. Tanabe, A. Fukunaga, Reviewing and benchmarking parameter control methods in differential evolution, IEEE Trans. Cybern. 50 (3) (2020) 1170–1184.
[27] R.D. Al-Dabbagh, F. Neri, N. Idris, M.S. Baba, Algorithmic design issues in adaptive differential evolution schemes: review and taxonomy, Swarm Evol. Comput. 43 (2018) 284–311.
[28] Z. Meng, C. Yang, Hip-DE: historical population based mutation strategy in differential evolution with parameter adaptive mechanism, Inf. Sci. (Ny) 562 (2021) 44–77.
[29] R.A. Sarker, S.M. Elsayed, T. Ray, Differential evolution with dynamic parameters selection for optimization problems, IEEE Trans. Evol. Comput. 18 (5) (2014) 689–707.
[30] Y. Zhou, W. Yi, L. Gao, X. Li, Adaptive differential evolution with sorting crossover rate for continuous optimization problems, IEEE Trans. Cybern. 47 (9) (2017) 2742–2753.
[31] G. Sun, J. Peng, R. Zhao, Differential evolution with individual-dependent and dynamic parameter adjustment, Soft. comput. 22 (17) (2018) 5747–5773.
[32] J. Zhang, A.C. Sanderson, JADE: adaptive differential evolution with optional external archive, IEEE Trans. Evol. Comput. 13 (5) (2009) 945–958.
[33] Z. Meng, J.S. Pan, HARD-DE: hierarchical archive based mutation strategy with depth information of evolution for the enhancement of differential evolution on numerical optimization, IEEE Access 7 (2019) 12832–12854.
[34] A. Ghosh, S. Das, A.K. Das, L. Gao, Reusing the past difference vectors in differential evolution—a simple but significant improvement, IEEE Trans. Cybern. 50 (11) (2020) 4821–4834.
[35] U. Halder, S. Das, D. Maity, A cluster-based differential evolution algorithm with external archive for optimization in dynamic environments, IEEE Trans. Cybern. 43 (3) (2013) 881–897.
[36] W. Yu, et al., Differential evolution with two-level parameter adaptation, IEEE Trans. Cybern. 44 (7) (2014) 1080–1099.
[37] Y. Wang, Z.-.Z. Liu, J. Li, H.-.X. Li, G.G. Yen, Utilizing cumulative population distribution information in differential evolution, Appl. Soft. Comput. 48 (2016) 329–346.
[38] W. Gong, A. Zhou, Z. Cai, A multioperator search strategy based on cheap surrogate models for evolutionary optimization, IEEE Trans. Evol. Comput. 19 (5) (2015) 746–758.
[39] S. Das, A. Mandal, R. Mukherjee, An Adaptive Differential Evolution Algorithm for Global Optimization in Dynamic Environments, IEEE Trans Cybern 44 (6) (2014) 966–978.
[40] Y. Cai, J. Wang, Differential evolution with neighborhood and direction information for numerical optimization, IEEE Trans Cybern 43 (6) (2013) 2202–2215.
[41] L. Tang, Y. Dong, J. Liu, Differential evolution with an individual-dependent mechanism, IEEE Trans. Evol. Comput. 19 (4) (2015) 560–574.
[42] A.K. Qin, V.L. Huang, P.N. Suganthan, Differential evolution algorithm with strategy adaptation for global numerical optimization, IEEE Trans. Evol. Comput. 13 (2) (2009) 398–417.
[43] Z.H. Zhan, Z.J. Wang, H. Jin, J. Zhang, Adaptive distributed differential evolution, IEEE Trans. Cybern. 50 (11) (2020) 4633–4647.
[44] S.M. Elsayed, R.A. Sarker, D.L. Essam, Differential evolution with multiple strategies for solving CEC2011 real-world numerical optimization problems, IEEE Congress Evolut. Comput. (2011) 1041–1048.
[45] S. Das, A. Konar, U.K. Chakraborty, Two improved differential evolution schemes for faster global search, in: Proceedings of the Conference on Genetic and Evolutionary Computation, 2005, pp. 991–998.
[46] R. Tanabe, A. Fukunaga, Success-History Based Parameter Adaptation for Differential Evolution, IEEE Congress Evolut. Comput. (2013) 71–78.
[47] J. Liu, J. Lampinen, A Fuzzy Adaptive Differential Evolution Algorithm, Soft. Comput. 9 (6) (2005) 448–462.
[48] M.M. Ali, A. Törn, Population set-based global optimization algorithms: some modifications and numerical studies, Comput. Oper. Res. 31 (10) (2004) 1703–1725.
[49] N.H. Awad, M.Z. Ali, J.J. Liang, Problem Definitions and Evaluation Criteria For the CEC 2017 Special Session and Competition on Single Objective Real-parameter Numerical Optimization, Nanyang Technological University, Singapore, 2016. Tech. Rep.
[50] A. Mohamed, A.A. Hadi, A.K. Mohamed, P. Agrawal, A. Kumar, and P. Suganthan, "Problem definitions and evaluation criteria for the CEC 2021 special session and competition on single objective bound constrained numerical optimization," Technical Report Nanyang Technological University Singapore, 2020.
[51] S. Das, P.N. Suganthan, Problem Definitions and Evaluation Criteria For CEC 2011 Competition on Testing Evolutionary Algorithms on Real World Optimization Problems, Jadavpur University, Nanyang Technological University, Kolkata, 2010, pp. 341–359.
[52] Q. Yang, et al., Stochastic Cognitive dominance leading particle swarm optimization for multimodal problems, Mathematics 10 (5) (2022) 761.
[53] S. Gao, Y. Yu, Y. Wang, J. Wang, J. Cheng, M. Zhou, Chaotic local search-based differential evolution algorithms for optimization, IEEE Trans. Syst. Man. Cybern. Syst. 51 (6) (2021) 3954–3967.
[54] Q. Yang, et al., Adaptive multimodal continuous ant colony optimization, IEEE Trans. Evol. Comput. 21 (2) (2017) 191–205.
[55] X. Xia, et al., NFDDE: a novelty-hybrid-fitness driving differential evolution algorithm, Inf Sci (Ny) 579 (2021) 33–54.
[56] Q. Yang, et al., An adaptive covariance scaling estimation of distribution algorithm, Mathematics 9 (24) (2021) 3207.
[57] L. Deng, C. Li, R. Han, L. Zhang, L. Qiao, TPDE: a tri-population differential evolution based on zonal-constraint stepped division mechanism and multiple adaptive guided mutation strategies, Inf Sci (Ny) 575 (2021) 22–40.
[58] A. Kumar, P.P. Biswas, P.N. Suganthan, Differential evolution with orthogonal array-based initialization and a novel selection strategy, Swarm Evol Comput 68 (2022), 101010.
[59] B.Y. Qu, P.N. Suganthan, J.J. Liang, Differential evolution with neighborhood mutation for multimodal optimization, IEEE Trans. Evol. Comput. 16 (5) (2012) 601–614.
[60] M.G. Epitropakis, D.K. Tasoulis, N.G. Pavlidis, V.P. Plagianakos, M.N. Vrahatis, Enhancing differential evolution utilizing proximity-based mutation operators, IEEE Trans. Evol. Comput. 15 (1) (2011) 99–119.
[61] A.W. Mohamed, A.A. Hadi, K.M. Jambi, Novel mutation strategy for enhancing SHADE and LSHADE algorithms for global numerical optimization, Swarm Evol. Comput. 50 (2019), 100455.
[62] Y. Cai, J. Wang, J. Yin, Learning-enhanced differential evolution for numerical optimization, Soft. comput. 16 (2) (2012) 303–330.

[63] Y. Cai, D. Wu, Y. Zhou, S. Fu, H. Tian, Y. Du, Self-organizing neighborhood-based differential evolution for global optimization, Swarm Evol. Comput. 56 (2020), 100699.
[64] J. Cheng, Z. Pan, H. Liang, Z. Gao, J. Gao, Differential evolution algorithm with fitness and diversity ranking-based mutation operator, Swarm Evol. Comput. 61 (2021), 100816, 2021/03/01/.
[65] X. Zhao, G. Xu, L. Rui, D. Liu, H. Liu, J. Yuan, A failure remember-driven self-adaptive differential evolution with top-bottom strategy, Swarm Evol. Comput. 45 (2019) 1–14.
[66] W. Gong, Z. Cai, Differential evolution with ranking-based mutation operators, IEEE Trans. Cybern. 43 (6) (2013) 2066–2081.
[67] L. Zou, Z. Pan, Z. Gao, J. Gao, Improving the search accuracy of differential evolution by using the number of consecutive unsuccessful updates, Knowl Based Syst. 250 (2022), 109005.
[68] A.W. Mohamed, P.N. Suganthan, Real-parameter unconstrained optimization based on enhanced fitness-adaptive differential evolution algorithm with novel mutation, Soft. comput. 22 (10) (2018) 3215–3235.
[69] Z. Meng, C. Yang, X. Li, Y. Chen, Di-DE: depth information-based differential evolution with adaptive parameter control for numerical optimization, IEEE Access 8 (2020) 40809–40827.
[70] A.A. Hadi, A.W. Mohamed, K.M. Jambi, LSHADE-SPA memetic framework for solving large-scale optimization problems, Complex Intell. Syst. 5 (1) (2019) 25–40.
[71] L. Gui, et al., A multi-role based differential evolution, Swarm Evol. Comput. 50 (2019), 100508.
[72] Y. Li, S. Wang, B. Yang, An improved differential evolution algorithm with dual mutation strategies collaboration, Expert Syst. Appl. 153 (2020), 113451.
[73] Y. Wang, H.-.X. Li, T. Huang, L. Li, Differential evolution based on covariance matrix learning and bimodal distribution parameter setting, Appl. Soft. Comput. 18 (2014) 232–247.
[74] A. Ghosh, et al., Using spatial neighborhoods for parameter adaptation: an improved success history based differential evolution, Swarm Evol. Comput. 71 (2022), 101057.
[75] Q. Yang, et al., A distributed swarm optimizer with adaptive communication for large-scale optimization, IEEE Trans. Cybern. 50 (7) (2020) 3393–3408.
[76] Q. Yang, W.N. Chen, J.D. Deng, Y. Li, T. Gu, J. Zhang, A level-based learning swarm optimizer for large-scale optimization, IEEE Trans. Evol. Comput. 22 (4) (2018) 578–594.
[77] A.W. Mohamed, A.A. Hadi, A.M. Fattouh, K.M. Jambi, LSHADE with semi-parameter adaptation hybrid with CMA-ES for solving CEC 2017 benchmark problems, IEEE Congress Evolut. Comput. (2017) 145–152.
[78] Q. Yang, X. Guo, X.-.D. Gao, D.-.D. Xu, Z.-.Y. Lu, Differential elite learning particle swarm optimization for global numerical optimization, Mathematics 10 (8) (2022) 1261.
[79] J. Brest, M.S. Maučec, B. Bošković, Single objective real-parameter optimization: algorithm jSO, in: IEEE Congress on Evolutionary Computation, 2017, pp. 1311–1318.
[80] Q. Yang, et al., Stochastic triad topology based particle swarm optimization for global numerical optimization, Mathematics 10 (7) (2022) 1032.
[81] A.W. Mohamed, A.K. Mohamed, Adaptive guided differential evolution algorithm with novel mutation for numerical optimization, Int J. Mach. Learn Cybern. 10 (2) (2019) 253–277.
[82] N.H. Awad, M.Z. Ali, P.N. Suganthan, Ensemble sinusoidal differential covariance matrix adaptation with euclidean neighborhood for solving CEC2017 benchmark problems, IEEE Congress Evolut. Comput. (2017) 372–379.
[83] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, Swarm Evol. Comput. 1 (1) (2011) 3–18.
[84] R. Tanabe, A.S. Fukunaga, Improving the search performance of SHADE using linear population size reduction, in: IEEE Congress on Evolutionary Computation, 2014, pp. 1658–1665.